

Konzeption und Bewertung des Technologie-Templates „Template-Projekte“

Technische Untersuchung einer wiederverwendbaren Basis für Web-, Cloud- und
Desktop-Projekte

Projekt: Template-Projekte
Verfasser: Marcel Tenhaft
Dokumentstand: 9. August 2026

Inhaltsverzeichnis

Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
Abkürzungsverzeichnis	VI
1 Einleitung	1
1.1 Ausgangslage	1
1.2 Problemstellung	1
1.3 Zielsetzung	1
1.4 Fragestellungen	2
1.5 Methodisches Vorgehen	2
1.6 Aufbau der Fallstudie	3
2 Anforderungen an das Technologie-Template	4
2.1 Zielbild des Technologie-Templates	4
2.2 Anforderungen an Wiederverwendbarkeit und Standardisierung	4
2.3 Anforderungen für Web-, Cloud- und Desktop-Anwendungen	5
2.4 Qualitäts- und Wartbarkeitsanforderungen	5
2.5 Abgrenzung	6
3 Architektur des Technologie-Templates	6
3.1 Gesamtarchitektur	6
3.2 Frontend mit Vite und TypeScript	8
3.3 Backend mit FastAPI	9
3.4 Desktop-Integration mit Tauri und Rust	10
3.5 Shared Contracts und Schnittstellen	11
3.6 Konfigurationsmodell	11
3.7 Persistenz mit PostgreSQL, SQLAlchemy und Alembic	12
4 Projektprofile und Feature-Modell	13
4.1 Grundprinzip des Profilmodells	13
4.2 Profil Web-only	16
4.3 Profil Web-cloud	17

4.4	Profil Desktop-local	17
4.5	Profil Desktop-cloud	17
4.6	Profil Full-platform	18
4.7	Optionale Capabilities	18
4.8	Single-Repository-Strategie	19
5	Tooling und Entwicklungsworkflow	19
5.1	Rolle von control.py	19
5.2	Projektinitialisierung und Generierung	20
5.3	Systemprüfung und Installation	22
5.4	Entwicklungsbetrieb	22
5.5	Tests und Builds	23
5.6	Release- und Packaging-Workflow	24
5.7	Entwickler-Onboarding	25
6	Qualitätssicherung und Verifikation	26
6.1	Teststrategie	26
6.2	Continuous Integration	27
6.3	Abnahme und technische Verifikation	28
6.4	Reproduzierbarkeit	29
6.5	Security und Konfigurationsgrenzen	30
6.6	Extern verbleibende Prüfungen	30
7	Bewertung des Technologie-Templates	31
7.1	Produktivitätswirkung	31
7.2	Stärken	32
7.3	Schwächen und Grenzen	33
7.4	Wartungsaufwand	34
7.5	Vergleich mit alternativen Template-Strategien	35
7.6	Gesamtbewertung	36
8	Einsatzmöglichkeiten und Transfer auf Kundenprojekte	37
8.1	Geeignete Projekttypen	37
8.2	Ungeeignete und eingeschränkt geeignete Projekttypen	39
8.3	Analyse von Kundenanforderungen	40

8.4	Auswahl des Projektprofils	41
8.5	Generierung eines Kundenprojekts	41
8.6	Überführung in die Produktentwicklung	42
9	Schlussbetrachtung	43
9.1	Zusammenfassung der Ergebnisse	43
9.2	Beantwortung der Fragestellungen	43
9.3	Kritische Würdigung	44
9.4	Ausblick	44
9.5	Fazit	45
	Literaturverzeichnis	46

Abbildungsverzeichnis

1	Methodische Schritte der Fallstudie	3
2	Gesamtarchitektur des Technologie-Templates	7
3	Komponenten und Verantwortungsgrenzen im Master Repository	7
4	Profilabhängige Laufzeitarchitektur	8
5	Providerneutrale Deployment-Architektur	10
6	Konfigurationspriorität und Sicherheitsgrenzen	12
7	Ableitung eines Produktprojekts aus einer gemeinsamen Template-Basis	14
8	Entscheidungsfluss für Profil und anschließende Capability-Auswahl	15
9	Strukturelle Feature-Zuordnung der fünf Profil-Presets	16
10	Implementierte Abhängigkeiten des Feature-Katalogs	18
11	Befehlsgruppen des zentralen Projekt-Toolings	20
12	Workflow der profilgesteuerten Projektgenerierung	21
13	Grundstruktur des Entwicklungsworkflows	23
14	Plattformübergreifendes Modell für Desktop-Releases	25
15	Nachweiskette der technischen Verifikation	27
16	Profilverifikation nach Evidenzstand und Commit	28
17	Qualitative Eignungsübersicht nach Architekturfit und Anpassungsbedarf	38
18	Vertikaler Transferprozess von der Kundenanforderung zum Produktprojekt	42

Tabellenverzeichnis

1	Zentrale Anforderungen an das Technologie-Template	4
2	Anforderungsmatrix der unterstützten Projektarten	5
3	Deklarierte und aufgelöste Technologie-Stack im untersuchten Commit	9
4	Deklarierte Basisfeatures und resultierende Laufzeitverzeichnisse	16
5	Profilbezogene technische Verifikation	29
6	Offene und externe Verifikationspunkte auf a4487ac	31
7	Evidenzbasierte Gegenüberstellung von Stärken und Grenzen	33
8	Qualitativer Vergleich von Template-Strategien	35
9	Zusammengeführte Bewertung des Technologie-Templates	36
10	Zuordnung technischer Projekttypen zur Template-Baseline	38
11	Kompakte Checkliste zur technikneutralen Anforderungsanalyse	40

Abkürzungsverzeichnis

Abk.	Bedeutung
------	-----------

API	Application Programming Interface
AWS	Amazon Web Services
CI	Continuous Integration
CLI	Command Line Interface
CORS	Cross-Origin Resource Sharing
CSP	Content Security Policy
DB	Database (Datenbank)
DEB	Debian-Paketformat
E2E	End-to-End
GCP	Google Cloud Platform
HPC	High Performance Computing
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
PG	PostgreSQL
PID	Process Identifier
RBAC	Role-Based Access Control
SDK	Software Development Kit
SQL	Structured Query Language
TLS	Transport Layer Security
UI	User Interface
URL	Uniform Resource Locator
ZIP	komprimiertes Archivformat

1 Einleitung

1.1 Ausgangslage

Bei neuen Softwareprojekten kehren technische Entscheidungen wieder. Sie betreffen Architektur, Technologien, Schnittstellen, Konfiguration, Tests, Build und Deployment. Werden diese Grundlagen jeweils neu geschaffen, entstehen unterschiedliche technische Ausgangslagen. Zugleich benötigen Web-, Cloud- und Desktop-Anwendungen gemeinsame Strukturen und plattformspezifische Erweiterungen.

Das Projekt „Template-Projekte“ bündelt diese Grundlagen in einem wiederverwendbaren Full-Stack-Technologie-Template. Es soll eine gemeinsame Basis für Web-, Cloud- und Desktop-Projekte bereitstellen (kleiveist, 2026k). Eine solche Basis kann wiederkehrende Entscheidungen bündeln und ein konsistentes Entwicklungsmodell schaffen. Gemeinsame Bestandteile müssen dafür klar von plattformspezifischen Erweiterungen getrennt sein.

Kurz erklärt: Arten von Softwareprojekten

Ein Softwareprojekt umfasst die geplante Entwicklung oder Anpassung eines Computerprogramms. Eine Web-Anwendung wird meist im Browser geöffnet. Eine Cloud-Anwendung nutzt Dienste, die auf Servern bereitgestellt und über ein Netzwerk erreicht werden. Eine Desktop-Anwendung wird dagegen auf einem Computer ausgeführt und kann dessen Betriebssystem direkt einbeziehen. Das untersuchte Template soll für diese verschiedenen Projektarten eine gemeinsame technische Ausgangsbasis liefern.

1.2 Problemstellung

Die wiederholte Initialisierung ähnlicher Projekte bindet Aufwand außerhalb der Produktentwicklung. Ohne gemeinsame Baseline unterscheiden sich Struktur, Werkzeuge, Konfiguration und Build-Prozesse; getrennte Ausgangsstände erhöhen den Wartungsaufwand und können technisch auseinanderlaufen.

Das Template muss daher Standardisierung und Flexibilität verbinden: Eine gemeinsame Basis soll Wiederverwendung ermöglichen, ohne Projekte mit unnötigen Komponenten zu belasten. Untersucht wird, wie Projektprofile und optionale Erweiterungen diesen Zielkonflikt lösen und wo Anpassungsbedarf bleibt (kleiveist, 2026f, 2026g).

Kurz erklärt: Template und Baseline

Ein Template ist eine wiederverwendbare Vorlage, aus der neue Projekte abgeleitet werden können. Die Baseline ist der gemeinsame technische Ausgangsstand dieser Vorlage, etwa für Struktur, Werkzeuge und Abläufe. Dadurch müssen wiederkehrende Grundlagen nicht für jedes Projekt neu aufgebaut werden. Die einzelnen Projekte können die gemeinsame Basis anschließend passend zu ihrem Zweck erweitern.

1.3 Zielsetzung

Die Fallstudie untersucht Architektur, gemeinsame und profilspezifische Bestandteile sowie die Abläufe für Entwicklung und Qualitätssicherung im Projekt „Template-Projekte“ (kleiveist, 2026f, 2026l).

Anhand vorhandener Projektartefakte und Nachweise bewertet sie Wiederverwendbarkeit, Standardisierung, Produktivitätspotenzial und Eignung für verschiedene Projektarten. Daraus leitet sie Einsatz- und Auswahlkriterien, technische Grenzen und externe Voraussetzungen ab (kleiveist, 2026j). Nicht

untersucht werden eine universelle Eignung für alle Softwareklassen oder eine produktspezifische Facharchitektur.

Kurz erklärt: Technische Fallstudie

Eine technische Fallstudie untersucht ein konkretes Projekt systematisch und in seinem vorhandenen Zustand. Dafür werden hier unter anderem dessen Dateien, Abläufe und technische Nachweise betrachtet. So lassen sich belegte Stärken, Grenzen und offene Punkte voneinander unterscheiden. Die Ergebnisse gelten für den untersuchten Stand und beweisen nicht, dass das Template für jede Art von Software geeignet ist.

1.4 Fragestellungen

Sechs Fragen strukturieren Untersuchung und Bewertung:

1. Welche fachlich-technischen, qualitativen und betrieblichen Anforderungen werden durch das Technologie-Template adressiert?
2. Wie unterstützen die Architektur und das Profilmodell die Bereitstellung unterschiedlicher Varianten für Web-, Cloud- und Desktop-Projekte?
3. Inwiefern fördern der gemeinsame technische Kern und die Single-Repository-Strategie die Wiederverwendung und Standardisierung zwischen Projekten?
4. Welches Potenzial bietet das Template, wiederkehrende Aufwände bei Projektinitialisierung, Entwicklungsworkflow, Qualitätssicherung und Bereitstellung zu reduzieren?
5. Für welche Projektarten und Kundenanforderungen stellt das Template eine geeignete technische Ausgangsbasis dar?
6. Welche technischen Grenzen, Wartungsaufwände, Risiken und externen Abhängigkeiten sind bei seinem Einsatz zu berücksichtigen?

Ihre Beantwortung stützt sich auf Anforderungen, Architektur und vorhandene Verifikationsnachweise.

1.5 Methodisches Vorgehen

Die Untersuchung folgt einer technischen Fallstudie (Runeson & Höst, 2009). Eine Bestandsaufnahme erfasst Struktur, Komponenten und Verantwortungsgrenzen des Master-Repository. Die anschließende Architekturanalyse betrachtet Frontend, Backend, Desktop-Integration, Schnittstellen, Persistenz und Deployment. Sie trennt belegte Eigenschaften von Annahmen und offenen Prüfpunkten (kleiveist, 2026f).

Die Profilanalyse untersucht gemeinsamen Kern, Varianten und optionale Capabilities. Ergänzend wird das Python-Tooling entlang von Generierung, Systemprüfung, Installation, Entwicklung, Tests, Builds und Release betrachtet (kleiveist, 2026g, 2026l). Projektinterne Architektur-, Profil- und Werkzeugbeschreibungen dienen dabei als Primärquellen für den dokumentierten Stand.

Vorhandene Tests, Build-Ergebnisse und dokumentierte Abnahmen werden den Anforderungen und Architekturentscheidungen zugeordnet. Externe oder produktspezifische Prüfungen bleiben getrennt.

Diese Evidenz trägt die Bewertung und den Transfer auf mögliche Kundenprojekte (kleiveist, 2026j). Abbildung 1 zeigt die Abfolge der Untersuchung. Bewertung und Transfer folgen damit erst nach Bestandsaufnahme, Analyse und Verifikation; offene Prüfpunkte werden nicht als bestätigte Eigenschaften behandelt.

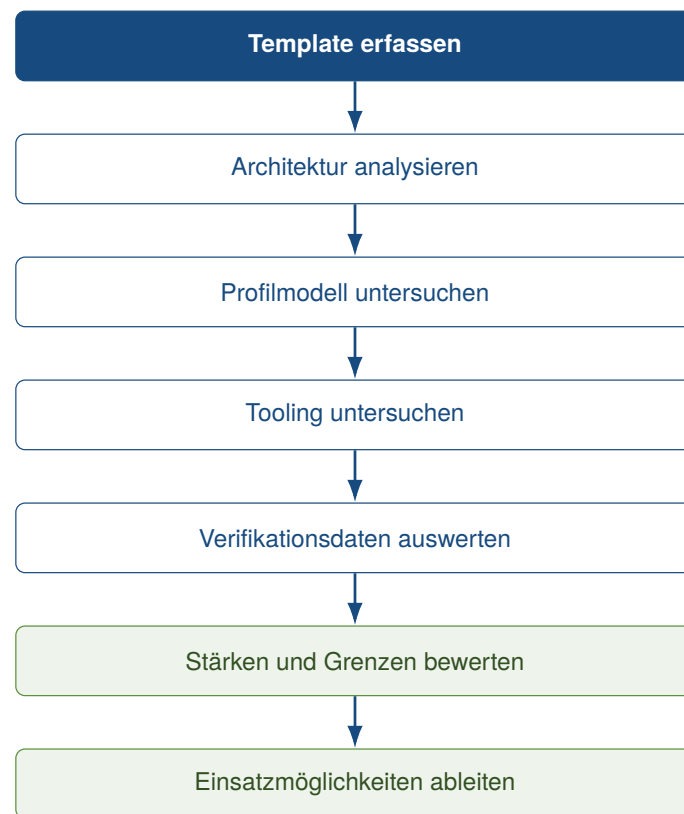


Abbildung 1: Methodische Schritte der Fallstudie

Quelle: Eigene Darstellung.

Kurz erklärt: Vorgehen der Untersuchung

Die Softwarearchitektur beschreibt den Aufbau eines Programms und das Zusammenspiel seiner Teile. Bei einer Analyse werden diese Teile und ihre Beziehungen geordnet untersucht. Verifikation bedeutet, eine technische Aussage durch eine passende Prüfung zu kontrollieren. Technische Evidenz ist der nachvollziehbare Nachweis dafür, beispielsweise ein Test- oder Build-Ergebnis. In dieser Fallstudie werden nicht geprüfte Punkte deshalb nicht wie bestätigte Eigenschaften behandelt.

1.6 Aufbau der Fallstudie

Auf die Anforderungen folgen Architektur, Projektprofile und Tooling. Danach behandelt die Fallstudie Qualitätssicherung und Verifikation, bewertet den festgestellten Stand und überträgt die Ergebnisse auf Kundenprojekte. Die Schlussbetrachtung beantwortet die Forschungsfragen und benennt Grenzen sowie Entwicklungsperspektiven.

2 Anforderungen an das Technologie-Template

2.1 Zielbild des Technologie-Templates

Das Template soll eine wiederverwendbare Basis für Web-, Cloud- und Desktop-Projekte bilden. Dafür verbindet es geplante Gemeinsamkeiten mit beherrschter Variabilität (Clements & Northrop, 2001; Krueger, 1992). Ein gemeinsamer Kern soll Struktur, Workflow, Tooling, Dokumentation, Konfiguration, Tests und Schnittstellen zentral bündeln.

Profile wählen daraus passende Kombinationen; optionale Capabilities wie PostgreSQL erweitern kompatible Backend-Profile (kleiveist, 2026g). Frontend, Backend, Desktop-Schicht, Persistenz, Tooling und Deployment bleiben getrennte Verantwortungsbereiche. Diese Aussagen beschreiben zunächst Anforderungen, keine Prüfergebnisse.

Kurz erklärt: Technische Verantwortungsbereiche

Das Frontend ist der sichtbare Teil einer Anwendung, den Menschen bedienen. Das Backend arbeitet im Hintergrund und verarbeitet Anfragen oder stellt Daten bereit. Eine Desktop-Schicht bindet die Anwendung an ein Betriebssystem und ermöglicht dort zusätzliche Funktionen. Persistenz sorgt dafür, dass Daten über das Beenden eines Programms hinaus erhalten bleiben. Deployment bezeichnet das Bereitstellen und Starten der Anwendung in ihrer Zielumgebung. Das Template soll diese Bereiche klar trennen; an dieser Stelle sind das zunächst Anforderungen und noch keine bestätigten Prüfergebnisse.

2.2 Anforderungen an Wiederverwendbarkeit und Standardisierung

Die Baseline soll gemeinsame Strukturen und öffentliche Arbeitsabläufe vorgeben, aber kontrollierte Profilvarianten zulassen. Eine zentrale Pflege vermeidet auseinanderlaufende Kopien. Tabelle 1 fasst die überprüfbaren Anforderungen zusammen.

Tabelle 1: Zentrale Anforderungen an das Technologie-Template

ID	Anforderung	Operationalisiertes Ziel	spätere Prüfbasis
A-01	Baseline	Grundstruktur muss generierbar sein.	Projektgenerierung
A-02	Struktur	Verantwortlichkeiten liegen an konsistenten Stellen.	Repository-Struktur
A-03	Profile	Nur vorgesehene Komponenten werden aktiviert.	Profilmodell
A-04	Workflows	Prüfen, Initialisieren, Installieren, Starten, Testen und Bauen besitzen gemeinsame Einstiegspunkte.	Tooling
A-05	Wartbarkeit	Gemeinsame Bestandteile werden zentral gepflegt.	Repository-Strategie
A-06	Nachvollziehbarkeit	Änderungen und Konfiguration sind versioniert und dokumentiert.	Versionsstand, Dokumentation
A-07	Erweiterbarkeit	Optionale Fähigkeiten besitzen dokumentierte Abhängigkeiten.	Capability-Modell

Quelle: Eigene Darstellung.

Die genannte Prüfbasis dient später der Bewertung dieser Soll-Kriterien.

Kurz erklärt: Gemeinsam statt jedes Mal neu

Wiederverwendung bedeutet, bereits vorbereitete Strukturen oder Abläufe in mehreren Projekten erneut einzusetzen. Standardisierung gibt diesen Projekten gemeinsame Regeln und Einstiegspunkte, damit sie ähnlich aufgebaut und bedient werden. Eine zentrale Baseline bündelt diese gemeinsamen Teile an einer Stelle, an der sie gepflegt werden können. Dadurch sollen getrennte Vorlagen nicht unbemerkt technisch auseinanderlaufen.

2.3 Anforderungen für Web-, Cloud- und Desktop-Anwendungen

Alle Varianten sollen Frontend-Basis, Struktur, Konfigurationslogik, Tests und Dokumentation teilen. Browserprojekte benötigen keine Desktop-Schicht und nur bei Bedarf ein Backend. Cloudvarianten erfordern API, Deployment-Grenzen sowie Health- und Readiness-Prüfung. Desktopvarianten verwenden dasselbe Frontend in einer nativen Laufzeit; kombinierte Varianten ergänzen Backend und gegebenenfalls Browserzugriff.

Tabelle 2: Anforderungsmatrix der unterstützten Projektarten

Anforderung	Web	Cloud	Desktop lokal	Desktop + Cloud
Frontend	erforderlich	erforderlich	erforderlich	erforderlich
Backend / API	optional	erforderlich	nicht grundsätzlich	erforderlich
Desktop-Laufzeit	nein	nein	erforderlich	erforderlich
Deployment-Grenze	optional	erforderlich	nein	erforderlich
Persistenz	optional	optional	produktspezifisch	optional

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026g).

PostgreSQL ist nur eine optionale Capability für backendfähige Profile, kein eigenes Plattformprofil (kleiveist, 2026g). Öffentliche Client-Konfiguration und serverseitige Werte einschließlich Geheimnissen sind zu trennen; eine bestimmte Cloud-Plattform wird nicht vorausgesetzt.

Kurz erklärt: Plattformen und API

Web, Cloud und Desktop bezeichnen hier die bereits eingeführten Browser-, servergestützten und lokal installierten Anwendungsformen. Full Stack bezeichnet eine Lösung, die sowohl das sichtbare Frontend als auch ein Backend umfasst. Eine API ist eine festgelegte Schnittstelle, über die Programme Anfragen und Daten austauschen. Die Profile des Templates kombinieren diese Bereiche unterschiedlich; nicht jede Variante benötigt deshalb ein Backend oder eine Datenbank.

2.4 Qualitäts- und Wartbarkeitsanforderungen

Die Anforderungen orientieren sich an überprüfbaren Merkmalen von ISO/IEC 25010 (ISO/IEC, 2023). Komponenten und Profile sollen klare Verantwortlichkeiten besitzen, getrennt testbar und durch dokumentierte Entscheidungen wartbar sein. Erweiterungen und Technologieaktualisierungen sollen keine Kopie der gesamten Baseline erfordern, bleiben aber risikobehaftet.

Gleiche Eingaben und Konfigurationen sollen nachvollziehbare Generierungs- und Build-Ergebnisse liefern. Dies stärkt die prüfbare Beziehung zwischen Quellstand und Artefakt (Lamb & Zacchioli, 2022). Zielartefakte sollen kontrolliert erzeugbar und Konfigurationsprioritäten verständlich sein; serverseitige Geheimnisse bleiben von Client-Konfiguration getrennt.

Kurz erklärt: Qualitätsziele

Testbarkeit bedeutet, dass sich ein Baustein mit klaren Prüfungen kontrollieren lässt, möglichst unabhängig von anderen Teilen. Wartbarkeit beschreibt, wie gut Software verstanden, geändert und aktualisiert werden kann. Reproduzierbarkeit bedeutet, dass gleiche festgelegte Eingaben und Einstellungen zu nachvollziehbaren Ergebnissen führen. Im Template sind dies Qualitätsziele, an denen die spätere technische Prüfung ausgerichtet wird.

2.5 Abgrenzung

Das Template ist keine universelle Softwareplattform. Sein Zielbereich umfasst wiederkehrende Projekte für Web, Backend und Desktop. Dazu zählen auch Full-Stack-Business-Anwendungen.

Geschäftsregeln, Domänen- und Kundendatenmodelle sowie produktspezifische Authentifizierungs-, Rollen- und Berechtigungskonzepte gehören nicht zur Baseline. Ein Cloud-Provider wird nicht vorgegeben (kleiveist, 2026f).

Stark plattformspezifische native Anwendungen, Spiele und hochspezialisierte Echtzeitsysteme liegen außerhalb des unmittelbaren Zielbereichs.

Kurz erklärt: Baseline statt fertigem Produkt

Der Scope legt fest, was eine Untersuchung oder Lösung einschließt und was außerhalb ihres Zielbereichs liegt. Produktspezifisch sind Funktionen, die erst für einen bestimmten Anwendungsfall oder Kunden benötigt werden. Die Baseline liefert gemeinsame technische Grundlagen, ist aber noch kein fertiges Produkt. Fachregeln, konkrete Datenmodelle und passende Zugriffsrechte müssen daher im jeweiligen Produktprojekt ergänzt werden.

3 Architektur des Technologie-Templates

3.1 Gesamtarchitektur

Die Baseline besteht aus einem Master-Repository mit gemeinsamem Kern und deklarativen Varianten. Profile wählen wiederverwendbare Features; optionale Capabilities erweitern nur kompatible Profile. Diese beherrschte Variabilität vermeidet getrennte Template-Kopien (Clements & Northrop, 2001; kleiveist, 2026g).

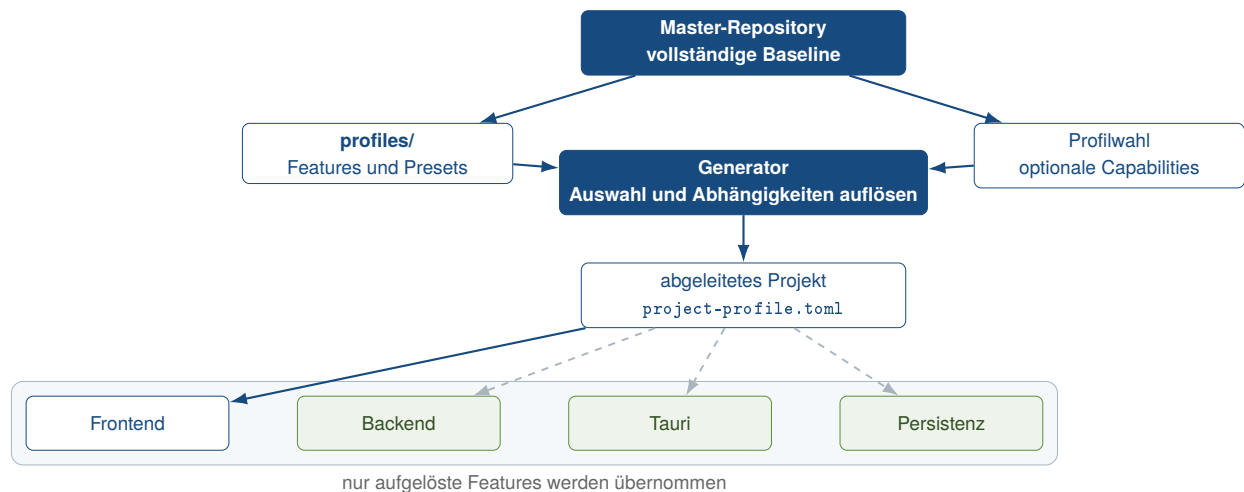


Abbildung 2: Gesamtarchitektur des Technologie-Templates

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026f) und kleiveist (2026g).

Abbildung 2 trennt die Baseline vom abgeleiteten Projekt. Der Generator löst Profil und Capabilities auf, übernimmt die zugehörigen Pfade und dokumentiert das Ergebnis in `project-profile.toml`. Frontend, Backend, Tauri und Persistenz bleiben eigenständige, teils optionale Bausteine (kleiveist, 2026f).

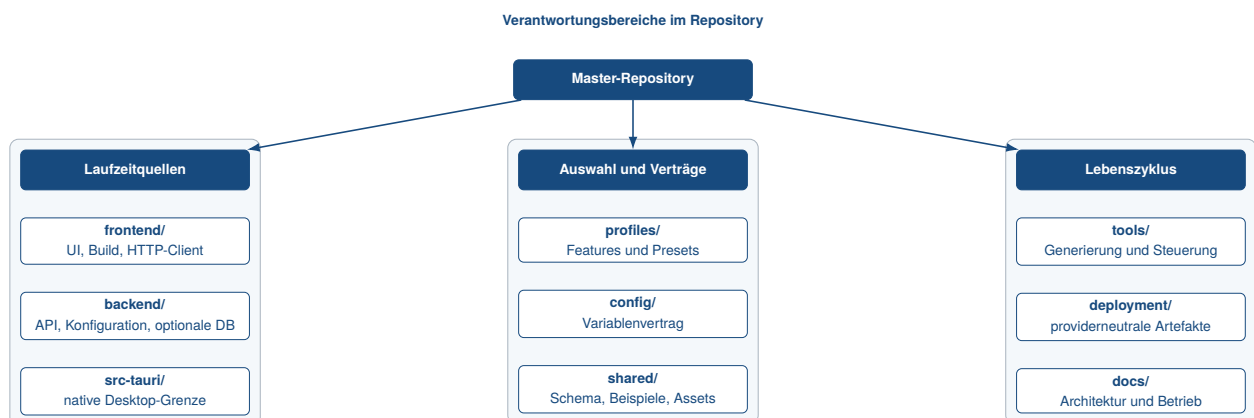


Abbildung 3: Komponenten und Verantwortungsgrenzen im Master Repository

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026f).

Die Komponententrennung in Abbildung 3 hält Client, Server, native Integration und gemeinsame Artefakte auseinander. Profile und Konfiguration beschreiben Struktur und Laufzeitwerte; Tooling und Deployment steuern die Komponenten außerhalb der Produktlaufzeit.

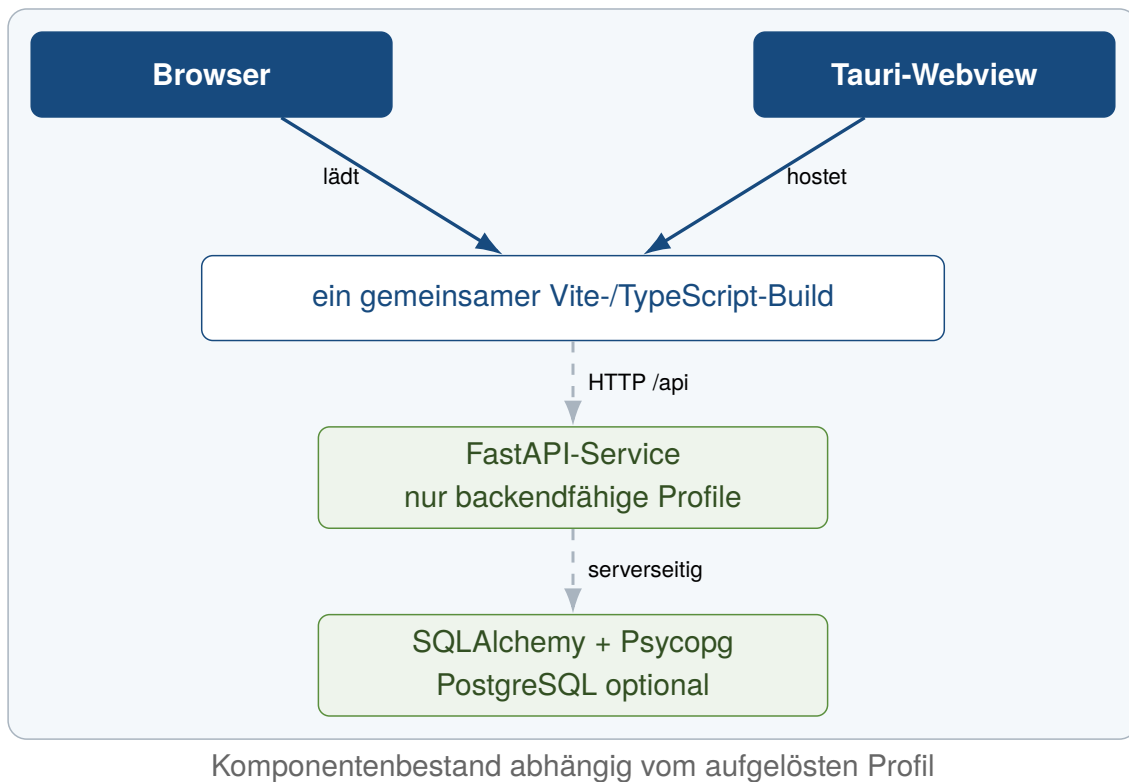


Abbildung 4: Profilabhängige Laufzeitarchitektur

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026f) und kleiveist (2026g).

Zur Laufzeit lädt entweder der Browser oder eine Tauri-Webview denselben Vite-Build (Abbildung 4). Backendfähige Profile kommunizieren über HTTP mit FastAPI; nur das Backend greift auf eine aktivierte Persistenz zu. So bleiben Client, Server und Datenhaltung getrennt test- und erweiterbar.

Kurz erklärt: Die Architekturabbildungen lesen

Eine Softwarearchitektur ist ein Bauplan, der Komponenten, ihre Aufgaben und ihre Verbindungen zeigt. Eine Komponente ist dabei ein abgegrenzter technischer Baustein. Ein Repository ist ein verwalteter Projektordner mit Änderungshistorie; das Master-Repository ist hier die zentrale Vorlage, aus der Projekte erzeugt werden. Der Client läuft auf der Seite der Nutzenden und enthält das sichtbare Frontend, während der Server das Backend im Hintergrund ausführt. Eine Datenbank speichert Daten dauerhaft und wird in passenden Profilen nur vom Backend angesprochen. Deployment bedeutet, diese Bausteine in einer Zielumgebung bereitzustellen und zu starten. Die Trennung von Verantwortlichkeiten sorgt dafür, dass jeder Baustein eine klare Aufgabe hat und möglichst eigenständig getestet oder geändert werden kann.

3.2 Frontend mit Vite und TypeScript

Das Frontend ist die gemeinsame Präsentationsschicht aller fünf Profile. Vite dient Entwicklung und Build; TypeScript prüft den strikt konfigurierten Quellcode, Vitest den HTTP-Client (Microsoft, 2026; VoidZero Inc. and Vite Contributors, 2026).

Das generierte Profilmul aktiviert Backendzugriffe nur in passenden Varianten. Browser und Tauri verwenden dieselben Quellen und eine öffentliche `VITE_API_BASE_URL`. Serverlogik, Geheimnisse und direkter Datenbankzugriff bleiben außerhalb des Clients; Web-only und Desktop-local benötigen kein

Backend (kleiveist, 2026f, 2026g).

Tabelle 3 dokumentiert Bereiche und Versionen des untersuchten Commits.

Tabelle 3: Deklarierter und aufgelöster Technologie-Stack im untersuchten Commit

Technologie	Rolle im Template	Version / Quelle	Projektbeleg
Vite	Entwicklungsserver und Frontend-Build	Bereich ab 6.0.7; Lock: 6.4.2	frontend/package.json, frontend/package-lock.json
TypeScript	statische Prüfung des Frontends	Bereich ab 5.7.3; Lock: 5.9.3	frontend/package.json, frontend/tsconfig.json
FastAPI	HTTP-API und Validierung	≥ 0.111 , < 1 ; Produktion: 0.111.0	backend/requirements.txt, backend/requirements-production.txt
Tauri	Desktop-Webview und Packaging-Grenze	Major 2; Lock: 2.11.5	src-tauri/Cargo.toml, src-tauri/Cargo.lock
Rust	native Kompositionsschicht	mindestens 1.77; Edition 2021	src-tauri/Cargo.toml
PostgreSQL	optionale relationale Laufzeiteinheit	Container: 16.4-alpine3.20	deployment/compose.yaml
SQLAlchemy	Engine-, Session- und Modellbasis	≥ 2 , < 3 ; Produktion: 2.0.36	backend/requirements-database*.txt
Alembic	explizite Schemamigrationen	≥ 1.13 , < 2 ; Produktion: 1.14.0	backend/requirements-database*.txt
Psycopg	PostgreSQL-Treiber	≥ 3.2 , < 4 ; Produktion: 3.2.3	backend/requirements-postgres*.txt

Quelle: Eigene Darstellung auf Grundlage des geprüften Projektstands (kleiveist, 2026k).

Kurz erklärt: Vite und TypeScript

Das Frontend ist die sichtbare und bedienbare Seite der Anwendung. JavaScript ist eine Sprache für solche Anwendungen; TypeScript erweitert JavaScript um Typangaben, mit denen bestimmte Fehler schon während der Entwicklung auffallen. Vite ist das Werkzeug, das einen Entwicklungsserver startet und später den Build erzeugt. Der Entwicklungsserver stellt die Anwendung beim Programmieren lokal bereit, während der Build die auslieferbaren Dateien in `frontend/dist` erstellt. Vitest führt automatisierte Tests für Frontend-Code aus. Der HTTP-Client schickt Anfragen an ein Backend und verwendet dazu eine URL, also dessen Adresse im Netzwerk. Im Template wird dieser Zugriff nur für Profile mit Backend aktiviert.

3.3 Backend mit FastAPI

Das Backend bildet eine separate HTTP-Schicht für Web-cloud, Desktop-cloud und Full-platform. FastAPI stellt unter `/api` derzeit nur `/health` und `/ready` bereit. Produktspezifische Dienste, Fachmodelle und Authentifizierung sind nicht vorgegeben (kleiveist, 2026f; Ramírez, 2026).

Liveness meldet den Prozesszustand. Bei aktiver Datenbankfähigkeit prüft Readiness zusätzlich die Verbindung und antwortet bei Fehlern mit HTTP 503, ohne interne Details offenzulegen. Router, Einstellungen und Abhängigkeit sind getrennt testbar; Service- und Domänenmodule ergänzt erst das Produkt.

Im Deployment bleiben die Einheiten getrennt (Abbildung 5): Der Vite-Build ist statisch bereitstellbar, FastAPI läuft in einem eigenen nicht privilegierten Container. Compose bildet diese providerneutrale Grenze lokal ab; PostgreSQL kommt nur explizit hinzu (kleiveist, 2026e).

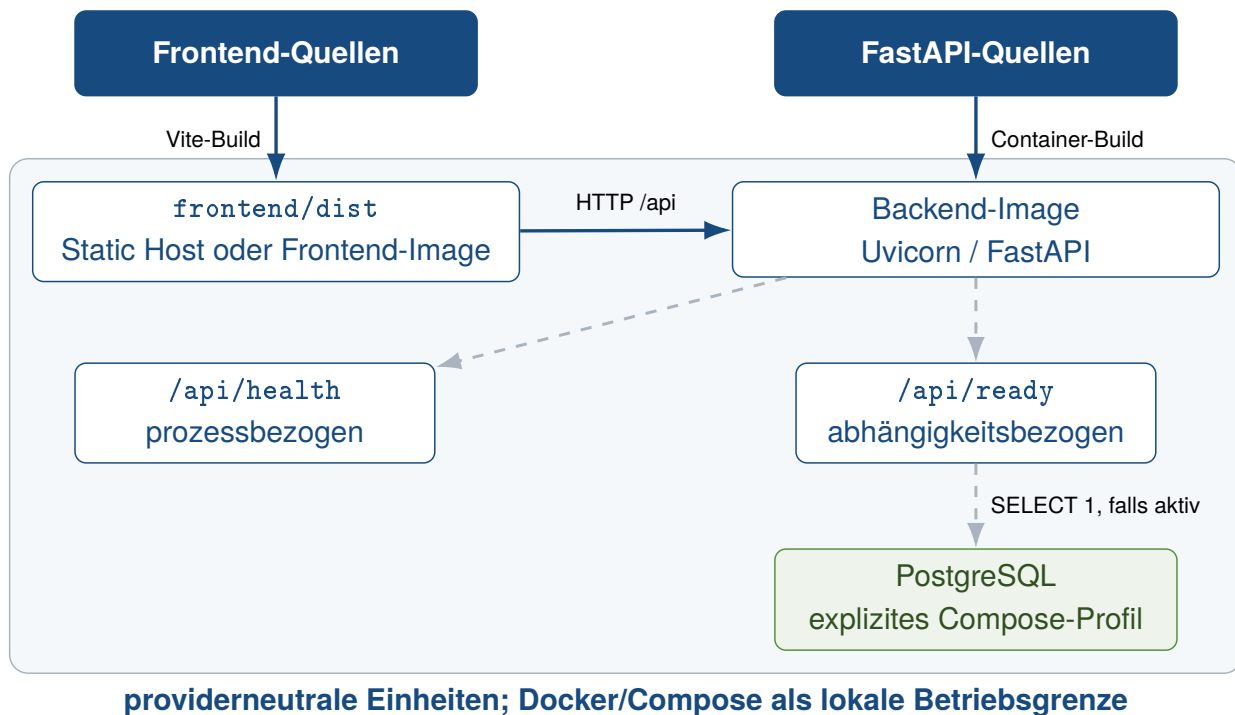


Abbildung 5: Providerneutrale Deployment-Architektur

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026e).

Kurz erklärt: FastAPI und HTTP

Das Backend ist der Teil im Hintergrund; hier wird es mit der Sprache Python und dem Werkzeug FastAPI erstellt. HTTP legt Regeln dafür fest, wie Programme Anfragen und Antworten über ein Netzwerk übertragen. Eine API ist eine vereinbarte Schnittstelle, über die das Frontend auf diesem Weg mit dem Backend kommuniziert. Eine Route oder ein Endpoint ist eine ansprechbare Adresse dieser API, wobei `/api` die gemeinsamen Prüfpfade gruppiert. `/api/health` meldet als Liveness-Prüfung, ob der Prozess lebt; `/api/ready` meldet als Readiness-Prüfung, ob er einsatzbereit ist, und prüft bei aktivierter Datenbank auch deren Verbindung. Ein HTTP-Statuscode fasst das Ergebnis als Zahl zusammen; HTTP 503 bedeutet hier, dass der Dienst vorübergehend nicht bereit ist. Ein Container verpackt einen Dienst mit seiner Laufzeit in eine getrennte Einheit, und das FastAPI-Backend läuft beim Deployment in einem solchen Container. Produktspezifische Dienste und eine Authentifizierung enthält diese Baseline noch nicht.

3.4 Desktop-Integration mit Tauri und Rust

Tauri bildet eine native Grenze, ohne das Frontend zu duplizieren. In der Entwicklung nutzt es den Vite-Server, im Build `frontend/dist`. Der Rust-Einstieg erstellt nur die Tauri-Anwendung; native Fachkommandos und ein FastAPI-Sidecar fehlen in der Baseline (Klabnik et al., 2026; kleiveist, 2026f; Tauri Contributors, 2026b).

Die Standard-Capability gewährt nur `core:default`; eine Content Security Policy begrenzt Ressourcen und Entwicklungsendpunkte. Weitere native Berechtigungen sind produktspezifisch zu ergänzen (Tauri Contributors, 2026a). Desktop-local bleibt im Frontend oder in späteren Tauri-Kommandos lokal; Desktop-cloud und Full-platform verwenden die öffentliche FastAPI-URL. Packaging, Signierung und die Wahl zwischen Remote-API, Sidecar oder lokalen Kommandos bleiben Produktentscheidungen.

Kurz erklärt: Tauri und die Desktop-Schicht

Tauri verwendet das vorhandene Web-Frontend und zeigt es in einer Webview, also einem eingebetteten Web-Anzeigebereich, als Desktop-Anwendung an. Ein Betriebssystem wie Windows, macOS oder Linux steuert den Computer; Rust bildet hier die native Anwendungsschicht, die direkt damit arbeiten und native Funktionen bereitstellen kann. Eine native Anwendung läuft installiert auf dem Computer und nicht ausschließlich in einem normalen Browser. Eine Capability beschreibt zusammen mit Berechtigungen, welche nativen Funktionen erlaubt sind; die Content Security Policy (CSP) begrenzt zusätzlich die nutzbaren Inhalte und Verbindungen. Ein Sidecar ist ein separates Hilfsprogramm, das zusammen mit der Anwendung ausgeliefert werden kann, in der Baseline aber nicht für FastAPI vorhanden ist. Eine Remote API läuft dagegen auf einem entfernten Server und wird über das Netzwerk erreicht. Packaging bündelt die Anwendung zu installierbaren Paketen, während Signing deren Herkunft und Unverändertheit kryptografisch bestätigt. Ob Remote API, Sidecar oder lokale Kommandos sowie Packaging und Signing verwendet werden, bleibt hier eine Produktentscheidung.

3.5 Shared Contracts und Schnittstellen

`shared/` gehört zu jedem generierten Projekt. Es enthält statische Assets, ein JSON-Schema nach Draft 2020-12 sowie gültige und ungültige Testbeispiele. Die Ablage ist serialisierbar und frameworkneutral, definiert aber weder eine Produkt-API noch ausführbaren Frameworkcode (kleiveist, 2026f, 2026k).

Die Laufzeit nutzt das Schema jedoch nicht automatisch: Weder Frontend noch Backend importieren es. Auch der Health-Vertrag ist als TypeScript-Interface und FastAPI-Antwort manuell gespiegelt. Ohne Codegenerierung oder Synchronisierung müssen Vertragsänderungen auf beiden Seiten und in den Konsumententests nachgezogen werden. Die dokumentierte Zielrolle gemeinsam genutzter Verträge ist damit noch nicht vollständig umgesetzt.

Kurz erklärt: Gemeinsame Datenregeln

Eine Schnittstelle ist eine festgelegte Grenze, an der zwei Programmteile Informationen austauschen. Ein Contract beschreibt dabei, welches Datenformat beide Seiten erwarten. JSON ist ein textbasiertes Format für strukturierte Daten, und ein JSON-Schema hält maschinenlesbare Regeln für deren Aufbau fest. Validierung prüft, ob konkrete Daten diese Regeln einhalten. Frontend und Backend müssen ihre Formate synchron halten, weil eine einseitige Änderung die Kommunikation sonst brechen kann. Codegenerierung kann aus einem gemeinsamen Contract passenden Code für beide Seiten erzeugen und so diese Abstimmung erleichtern. Im untersuchten Stand existieren Schema und Testbeispiele, sie werden zur Laufzeit aber nicht automatisch eingebunden; auch eine automatische Synchronisierung oder Codegenerierung ist noch nicht umgesetzt.

3.6 Konfigurationsmodell

Projektstruktur und Laufzeitkonfiguration sind getrennt. Das Projektmanifest bestimmt die Features und liegt in `project-profile.toml`. Der Variablenvertrag steht in `config/environment.toml`. Für aktive Features gilt die Priorität CLI-Override, Prozessumgebung, lokale `.env`, Vertragsdefault. Auch abgeleitete Werte bleiben nachvollziehbar (kleiveist, 2026i).

Abbildung 6 zeigt die Sicherheitsgrenzen. Vite erhält nur die freigegebene `VITE_API_BASE_URL`; FastAPI verarbeitet Serverwerte typisiert und maskiert `DATABASE_URL`. Das Tooling reicht nur relevante Werte

weiter und entfernt bekannte Secret-Namen aus der Tauri-Umgebung. Ungültige oder fehlende Werte führen zum kontrollierten Abbruch, ohne Geheimnisse auszugeben.

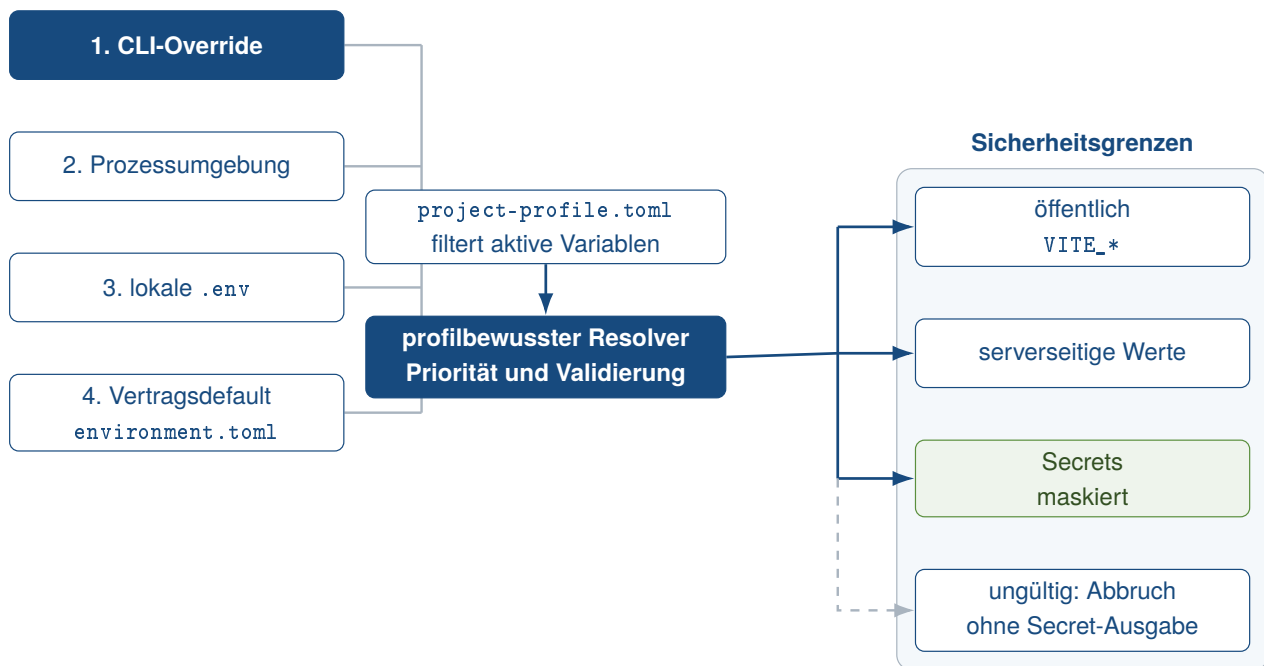


Abbildung 6: Konfigurationspriorität und Sicherheitsgrenzen

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026i).

Kurz erklärt: Konfiguration

Eine Konfiguration ist eine Sammlung von Einstellungen, die festlegt, wie und wo ein Programm ausgeführt wird. Eine Umgebungsvariable ist ein Wert, den die Laufzeitumgebung dem Programm bereitstellt; eine lokale `.env`-Datei kann solche Werte für die Entwicklung sammeln. `environment.toml` beschreibt die erlaubten Variablen und mögliche Defaults, also verwendete Standardwerte. `project-profile.toml` legt dagegen fest, welche technischen Bausteine zum Projekt gehören. Ein Override ersetzt einen niedriger priorisierten Wert bewusst, und der Resolver ermittelt daraus nach einer festen Reihenfolge den tatsächlich geltenden Wert. Ein Secret ist eine geheime Angabe wie ein Passwort und darf nicht an den sichtbaren Client gelangen. `DATABASE_URL` enthält die serverseitige Datenbankverbindung und wird maskiert. `VITE_API_BASE_URL` ist dagegen die öffentliche Backend-Adresse, die das Frontend verwenden darf.

3.7 Persistenz mit PostgreSQL, SQLAlchemy und Alembic

PostgreSQL ist optional und kein eigenes Profil. `postgres` setzt `database` und damit ein Backend voraus; `Web-only` und `Desktop-local` sind ausgeschlossen. Die Datenbankfähigkeit umfasst SQLAlchemy-Engine, Sessions und Alembic. PostgreSQL ergänzt Psycopg 3, URL-Vorgaben und Integrationstests und bleibt eine getrennte Laufzeiteinheit (kleiveist, 2026d; PostgreSQL Global Development Group, 2026).

Die Engine entsteht erst beim Zugriff und prüft Verbindungen vor der Nutzung; Sessions werden kontrolliert geschlossen. Die leere deklarative Base ist ein Erweiterungspunkt für Fachmodelle (SQLAlchemy Authors and Contributors, 2026).

Alembic nutzt dieselbe Metadatenbasis für Online- und Offline-Migrationen. Ohne Fachmodelle gibt es

keine initiale Revision. Weder `create_all()` noch automatische Migrationen beim FastAPI-Start sind implementiert; Änderungen erfordern explizite Alembic-Kommandos (Bayer, 2026; kleiveist, 2026d). `/api/health` bleibt datenbankunabhängig, `/api/ready` prüft bei aktiver Persistenz die Erreichbarkeit.

Kurz erklärt: Die Datenbankschicht

Eine Datenbank sorgt für Persistenz: Daten bleiben also auch nach dem Beenden des Programms erhalten. Eine relationale Datenbank ordnet Daten in miteinander verknüpften Tabellen; SQL ist die Sprache für Abfragen und Änderungen daran. PostgreSQL ist die optionale relationale Datenbank dieses Templates. Eine Verbindung ist der Kommunikationsweg zur Datenbank, und der Datenbanktreiber Psycopg stellt ihn zwischen Python und PostgreSQL bereit. SQLAlchemy vereinfacht den Zugriff: Die Engine verwaltet den technischen Zugang, eine Session bündelt die Arbeit mit Daten. Das Schema beschreibt die Struktur der Datenbank, etwa Tabellen und Spalten. Eine Migration ändert diese Struktur kontrolliert, und Alembic verwaltet solche Änderungen. Die Baseline enthält noch keine Fachmodelle oder erste Migration; Änderungen müssen ausdrücklich mit Alembic-Kommandos ausgeführt werden.

4 Projektprofile und Feature-Modell

4.1 Grundprinzip des Profilmodells

Das Master-Repository verwaltet Varianten featurebasiert und unterstützt damit systematische Wiederverwendung, ohne eine vollständige Software-Produktlinie zu bilden (Clements & Northrop, 2001; Krueger, 1992). Dabei bezeichnet *Core* die Pfade jedes Produktprojekts. Ein *Feature* bündelt Pfade und Abhängigkeiten einer technischen Fähigkeit, ein *Profil* eine zulässige Feature-Kombination. Eine *Capability* erweitert ein Profil optional und ist selbst kein Plattformprofil. Das *Manifest* dokumentiert die Auswahl im generierten Projekt (kleiveist, 2026g, 2026k). Zum Core gehören unter anderem Versionsstand, Konfiguration, Dokumentation, Profilbeschreibungen, gemeinsame Artefakte und Tooling. Technische Laufzeitkomponenten liegen dagegen in den jeweils ausgewählten Feature-Pfaden.

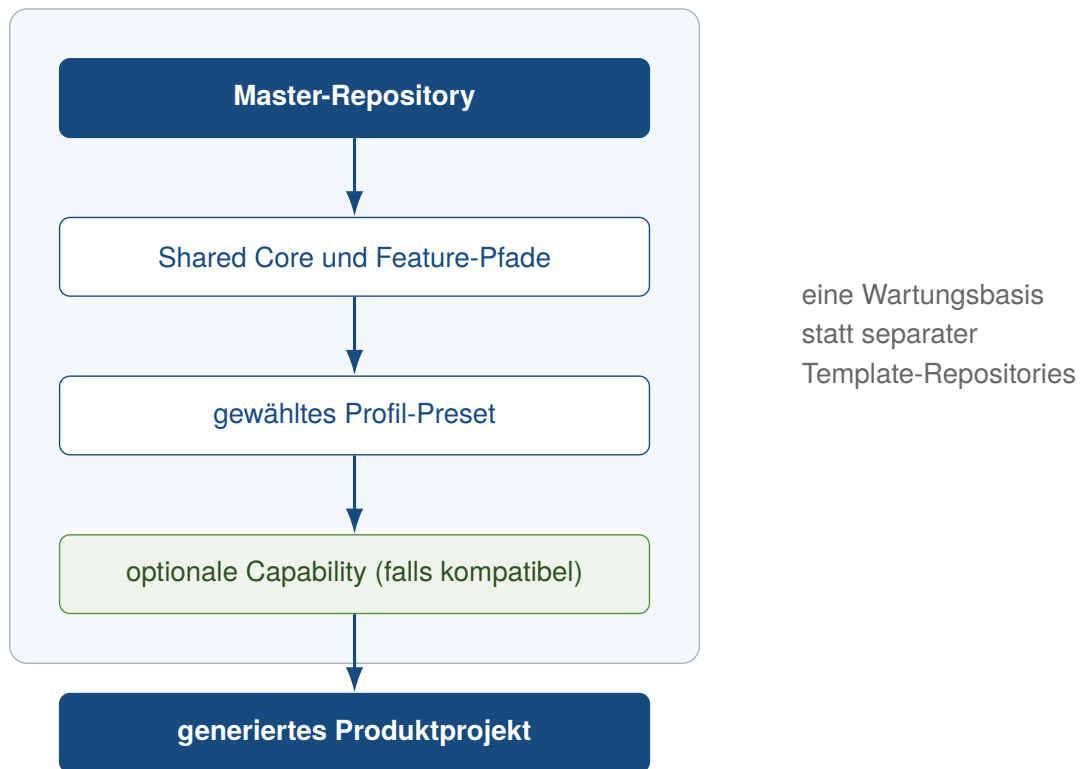


Abbildung 7: Ableitung eines Produktprojekts aus einer gemeinsamen Template-Basis

Der Generator löst Profil und kompatible Capabilities auf und kopiert Core und aktive Feature-Pfade in ein neues Produktprojekt (Abbildung 7). Nicht aktive Verzeichnisse werden nicht ausgewählt; Webprofile verlieren zusätzlich die Tauri-CLI. Name, Slug und bei Tauri die Anwendungs-ID sind anpassbar (kleiveist, 2026g, 2026l). Aus der Auflösung entsteht ein geordneter Scaffold-Plan. Dadurch enthält das Zielprojekt nicht lediglich deaktivierte Bausteine, sondern überwiegend nur die für seine Variante vorgesehenen Verzeichnisse.

Backend und Cloud treten in den aktuellen Presets gemeinsam auf. Bei Desktopprojekten unterscheidet ein zusätzlicher Browserkanal `full-platform` von `desktop-cloud`. Capabilities werden erst nach der Profilwahl geprüft; fehlende Plattformfeatures ergänzt der Resolver nicht stillschweigend (Abbildung 8).

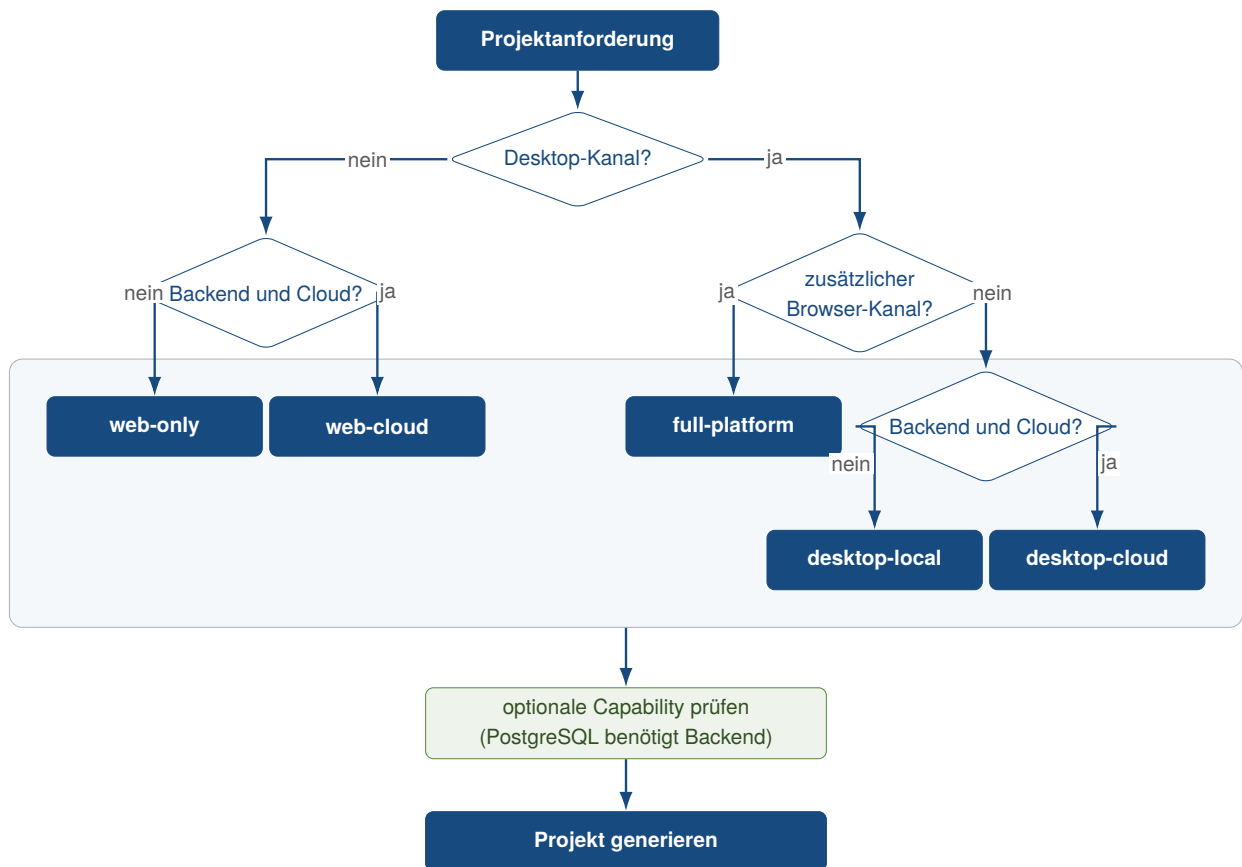


Abbildung 8: Entscheidungsfluss für Profil und anschließende Capability-Auswahl

Das erzeugte `project-profile.toml` hält Schema-Version, Profilidentität, angeforderte Erweiterungen und vollständig aufgelöste Features fest. Das Tooling validiert es und behandelt nur aktive Dienste und Workflows. Das Master-Manifest verwendet `full-platform` + `postgres`, ohne das Preset selbst zu verändern. Laufzeitwerte, Ports, URLs und Geheimnisse gehören dagegen zum getrennten Konfigurationsmodell (kleiveist, 2026i, 2026k). Im Manifest bleiben angeforderte optionale Features und das aufgelöste Ergebnis getrennt sichtbar. Diese Unterscheidung macht transitive Abhängigkeiten nachvollziehbar.

Abbildung 9 und Tabelle 4 vergleichen die fünf Presets. Keines aktiviert PostgreSQL vorab.

Profil	Frontend	FastAPI	Tauri	Cloud	PostgreSQL
web-only	enthalten	–	–	–	inkompatibel
web-cloud	enthalten	enthalten	–	enthalten	optional
desktop-local	enthalten	–	enthalten	–	inkompatibel
desktop-cloud	enthalten	enthalten	enthalten	enthalten	optional
full-platform	enthalten	enthalten	enthalten	enthalten	optional

Abbildung 9: Strukturelle Feature-Zuordnung der fünf Profil-Presets

Quelle: Eigene Darstellung auf Grundlage des Feature-Katalogs und der Profildefinitionen im geprüften Projektstand (kleiveist, 2026g, 2026k).

Tabelle 4: Deklarierte Basisfeatures und resultierende Laufzeitverzeichnisse

Profil	Basisfeatures	Laufzeitverzeichnisse	PostgreSQL
web-only	frontend	frontend/	inkompatibel
web-cloud	frontend, backend, cloud	frontend/, backend/, deployment/	optional
desktop-local	frontend, tauri	frontend/, src-tauri/	inkompatibel
desktop-cloud	frontend, backend, tauri, cloud	frontend/, backend/, src-tauri/, deployment/	optional
full-platform	frontend, backend, tauri, cloud	frontend/, backend/, src-tauri/, deployment/	optional

Kurz erklärt: Profilmodell

Der *Core* ist die gemeinsame Grundausstattung jedes erzeugten Projekts. Ein *Feature* ist ein einzelner technischer Baustein mit den dazugehörigen Dateien und benötigten Abhängigkeiten. Ein *Profil* ist wie eine vorkonfigurierte Ausstattung; ein *Preset* ist eine solche bereits festgelegte Auswahl. Eine *Capability* ergänzt ein Profil um eine optionale Fähigkeit. Das *Manifest* hält fest, was für ein Projekt ausgewählt und tatsächlich aufgelöst wurde. Der *Generator* erstellt das Projekt, während der *Resolver* zuvor die zulässige Kombination und ihre Voraussetzungen ermittelt. Der daraus entstehende *Scaffold* ist das Grundgerüst aus den benötigten Ordnern und Dateien. Eine Abhängigkeit ist eine Voraussetzung; bei einer transitiven Abhängigkeit wird eine weitere Voraussetzung nur indirekt über einen anderen Baustein benötigt.

4.2 Profil Web-only

web-only ergänzt den Core ausschließlich um das Vite-Frontend. Der Browser lädt es ohne FastAPI-, Desktop-, Deployment- oder Datenbankschicht; Tauri-Skript und -CLI werden entfernt. Das Profil eignet sich damit für reine Clientanwendungen ohne Backendzugriff (kleiveist, 2026g). Gegenüber web-cloud fehlen folglich API und Cloud-Feature; auch das generierte Frontend aktiviert keinen

Backend-Statusaufruf.

Kurz erklärt: web-only

`web-only` ist eine Anwendung für den Browser. Sie enthält das Vite-Frontend. Ein eigenes FastAPI-Backend, eine Desktop-Schicht, Deployment und eine Datenbank gehören nicht zu diesem Profil.

4.3 Profil Web-cloud

`web-cloud` kombiniert Vite-Frontend, getrenntes FastAPI-Backend und providerneutrale Deployment-Dateien. Eine native Desktop-Schicht fehlt (kleiveist, 2026e, 2026g). Datenbankmodul, Alembic und Psycopy kommen nur mit der kompatiblen, separat zu wählenden PostgreSQL-Capability hinzu. Das Frontend greift über eine öffentliche URL auf das getrennte Backend zu. Ohne Capability bleibt das Profil trotz Cloud-Bezeichnung datenbankfrei.

Kurz erklärt: web-cloud

`web-cloud` ist eine Browseranwendung mit einem getrennten FastAPI-Backend. Das Profil bringt außerdem eine anbieterneutrale Grundlage für das Deployment mit. Eine PostgreSQL-Datenbank kommt nur über die separat gewählte Capability hinzu.

4.4 Profil Desktop-local

`desktop-local` führt das gemeinsame Frontend in einer Tauri-Webview aus. Backend, Deployment und Datenbankpfade fehlen (kleiveist, 2026g). *Local* bezeichnet somit die fehlende Cloud-API, nicht eine eingebaute SQL-Datenbank. Da `database` ein Backend benötigt, ist PostgreSQL inkompatibel. Lokale Fachfunktionalität muss daher im Frontend oder in produktspezifisch ergänzten Tauri-Kommandos liegen.

Kurz erklärt: desktop-local

`desktop-local` führt das gemeinsame Frontend als Tauri-Anwendung auf dem eigenen Rechner aus. Eine zentrale Cloud-API und ein Template-Backend gehören nicht dazu. *Local* bedeutet deshalb nicht automatisch, dass eine lokale Datenbank vorhanden ist.

4.5 Profil Desktop-cloud

`desktop-cloud` verbindet das Tauri-Frontend über eine öffentliche URL mit dem getrennten FastAPI-Backend und ergänzt Deployment-Dateien. PostgreSQL ist kompatibel, bleibt aber optional (kleiveist, 2026e, 2026g).

Die technischen Basisfeatures entsprechen `full-platform`; die Profil-ID kennzeichnet jedoch den Desktop-Client als vorgesehenen Produktkanal. Die Abgrenzung ist semantisch: Ein zusätzlicher technischer Baustein entsteht daraus nicht.

Kurz erklärt: desktop-cloud

`desktop-cloud` ist eine Tauri-Desktopanwendung mit einem zentralen FastAPI-Backend. Der Desktop-Client erreicht dieses Backend über eine öffentliche URL. PostgreSQL ist möglich, bleibt aber eine optionale Erweiterung.

4.6 Profil Full-platform

`full-platform` stellt Browser und Tauri als zwei Produktkanäle derselben Frontend-Basis bereit; beide greifen auf dasselbe FastAPI-Backend zu. Von `desktop-cloud` unterscheidet sich der Scaffold nur in Profilidentität und Beschreibung, nicht in den Basisfeatures (kleiveist, 2026g, 2026k). Browser-Build und Desktopanwendung verwenden damit dieselbe Frontend- und Backendbasis.

Full schließt keine Persistenz ein. Erst `postgres` ergänzt die Datenbankfähigkeit.

Kurz erklärt: full-platform

`full-platform` bietet Browser und Desktop als zwei Zugangswege zur gleichen technischen Basis aus Frontend und Backend an. Beide verwenden dasselbe FastAPI-Backend. Eine Datenbank ist trotz des Namens nicht automatisch enthalten, sondern kommt erst mit `postgres` hinzu.

4.7 Optionale Capabilities

Der Katalog enthält sechs Features. `tauri` benötigt `frontend`; `cloud` und `database` benötigen `backend`; `postgres` benötigt `database`. Die gerichteten Regeln in Abbildung 10 beschreiben keine Laufzeitkommunikation (kleiveist, 2026d, 2026g).

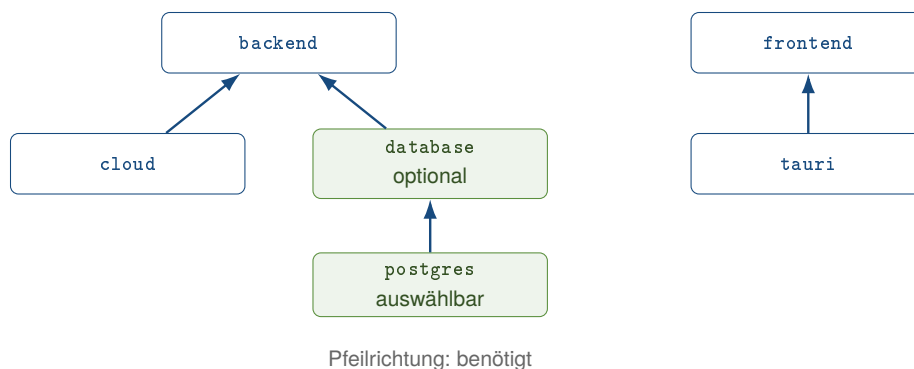


Abbildung 10: Implementierte Abhängigkeiten des Feature-Katalogs

Nur `postgres` ist derzeit als auswählbare Capability markiert. Für Backendprofile löst sie zunächst das optionale, intern bleibende `database` auf und ergänzt SQLAlchemy, Alembic, Psycopy sowie Tests. Der PostgreSQL-Dienst ist im Compose-Dokument bereits als inaktives Profil deklariert; die Capability ergänzt Backend-Infrastruktur und Abhängigkeiten (kleiveist, 2026e, 2026k). So besteht `web-cloud` + `postgres` aus Frontend, Backend, Cloud, Datenbankbasis und PostgreSQL-Erweiterung. Die Auswahl aktiviert also nicht erst den Compose-Eintrag, sondern die zugehörigen Backend-Pfade.

Der Resolver ergänzt nur optionale Voraussetzungen. Deshalb werden `web-only` + `postgres` und `desktop-local` + `postgres` wegen des fehlenden Backends abgewiesen. Neue Katalogeinträge gelten nicht allein durch ihre Deklaration als implementiert. Sie benötigen zusätzlich eigene Pfade, Abhängigkeiten und Tests.

Zwei Implementierungsabweichungen bleiben: Der Dialog bietet nur `selectable` PostgreSQL an, eine direkte `-with database`-Angabe akzeptiert jedoch die für Backendprofile optionale Datenbankfähigkeit. Zudem stammen Einträge für `.env.example` entgegen der allgemeinen Profildokumentation aus

`config/environment.toml`, nicht aus `features.toml`. Die Presets bleiben unverändert; die Trennung zwischen interner und auswählbarer Capability ist aber nicht vollständig durchgesetzt.

Kurz erklärt: optionale Fähigkeiten

Features können Voraussetzungen für andere Features sein; Tauri benötigt beispielsweise das Frontend. Eine optionale Capability wird nur auf Wunsch ergänzt, und *selectable* bedeutet, dass sie regulär zur Auswahl angeboten wird. Derzeit ist nur PostgreSQL auf diese Weise auswählbar. PostgreSQL braucht die Datenbankbasis, und diese wiederum braucht ein Backend, weil dort der Datenbankzugriff ausgeführt wird. Deshalb sind `web-only + postgres` und `desktop-local + postgres` nicht zulässig. Der Resolver prüft solche Abhängigkeiten, kann aber ein fehlendes Plattformfeature nicht eigenmächtig ergänzen.

4.8 Single-Repository-Strategie

Die fünf Profile sind keine Template-Forks. Core, Feature-Implementierungen, Profile und Generator liegen in einer Wartungsbasis, aus der eigenständige Produktprojekte entstehen (kleiveist, 2026g, 2026k).

Das Master enthält auch optionale Implementierungen und aktiviert selbst PostgreSQL. Ein generiertes Projekt erhält dagegen nur Core, aufgelöste Feature-Pfade, Manifest, Frontend-Profil und passende `.env.example`. Es bleibt danach vom unveränderten Master getrennt. Die zentrale Pflege reduziert parallele Baselines; eine automatische Rücksynchronisierung generierter Projekte besteht nicht. Diese Trennung schützt die Wartungsbasis vor Produktänderungen. Umgekehrt profitieren bereits erzeugte Produkte nicht ohne gezielte Übernahme von späteren Korrekturen des Masters.

Kurz erklärt: eine gemeinsame Wartungsbasis

Single Repository bedeutet hier, dass Core, Features, Profile und Generator gemeinsam an einer Stelle gepflegt werden. Dieses Master-Repository ist die maßgebliche *Source of Truth*, also der verlässliche Ausgangsstand der Vorlage. Die Profile sind keine getrennten Forks oder Kopien dieser Wartungsbasis. Der Generator erstellt daraus ein eigenständiges Produktrepository mit den ausgewählten Teilen. Danach bleiben Master und Produktprojekt getrennt. Spätere Updates des Masters werden nicht automatisch synchronisiert, sondern müssen gezielt in das Produkt übernommen werden.

5 Tooling und Entwicklungsworkflow

5.1 Rolle von `control.py`

`tools/control.py` bündelt die öffentlichen Arbeitsabläufe. Seine Befehle decken Initialisierung, Diagnose, Installation, Entwicklung, Konfiguration, Datenbank, Tests, Builds, Container, Tauri, Versionierung, Release und Dokumentation ab. Hilfe und Gruppenansichten strukturieren diese Oberfläche (kleiveist, 2026a, 2026l). Zu den wichtigsten Einstiegen gehören `init`, `doctor`, `install`, `run`, `test` und `build`. Die Gruppen `db`, `tauri`, `container` und `release` ergänzen nur die jeweils benötigten Fachabläufe.

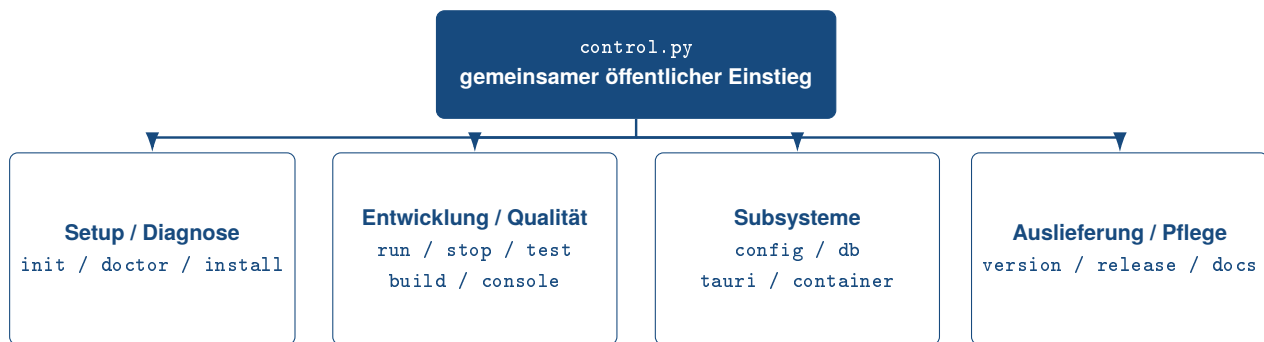


Abbildung 11: Befehlsgruppen des zentralen Projekt-Toolings

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026a) und kleiveist (2026l).

Die Handler laden Manifest und bei Bedarf Laufzeitkonfiguration und delegieren an die jeweiligen Fachwerkzeuge. Diese bleiben eigenständig. Die geführte `console` ruft dieselben Befehle auf und bildet kein zweites System.

Unterschiedliche Exitcodes trennen Hilfe und Erfolg, fachliche Fehler, Bedienfehler und Abbruch. Erwartete Profil- und Generierungsfehler erscheinen ohne Traceback; unerwartete Exceptions werden protokolliert. Diese Konvention unterstützt interaktive Diagnose und CI-Auswertung (kleiveist, 2026a, 2026c). Insbesondere lassen sich erfolgreiche Ausführung, fachlicher Fehlschlag und fehlerhafte Bedienung maschinell unterscheiden. Warnungen blockieren den Workflow nicht automatisch.

Kurz erklärt: control.py

`tools/control.py` ist ein Python-Skript und dient als Kommandozentrale des Templates. Ein Terminal ist ein textbasiertes Fenster, in dem Programme durch Eingaben gesteuert werden. Dort stellt das Skript eine *Command Line Interface* (CLI), also eine Kommandozeilenoberfläche, bereit. Ein Kommando wie `run` wählt eine Aufgabe; ein Argument liefert dazu eine zusätzliche Angabe. *Tooling* bezeichnet die Hilfswerkzeuge für Abläufe wie Installation, Tests und Builds. Ein Exitcode ist eine Zahl, mit der das Skript Erfolg, einen fachlichen Fehler oder eine falsche Bedienung meldet. Ein Traceback zeigt bei einem unerwarteten Python-Fehler die Kette der aufgerufenen Programmstellen; erwartete Bedien- und Profilfeiler werden hier ohne einen solchen technischen Detailbericht ausgegeben.

5.2 Projektinitialisierung und Generierung

`init` erzeugt aus Profil und kompatiblen Capabilities ein eigenständiges Produktprojekt. Die Auswahl kann geführt oder nicht interaktiv erfolgen; Projektidentität, Ziel und Dry-Run sind konfigurierbar. Eine geänderte Tauri-Identität erfordert einen gültigen Reverse-Domain-Identifizier (kleiveist, 2026g, 2026l).

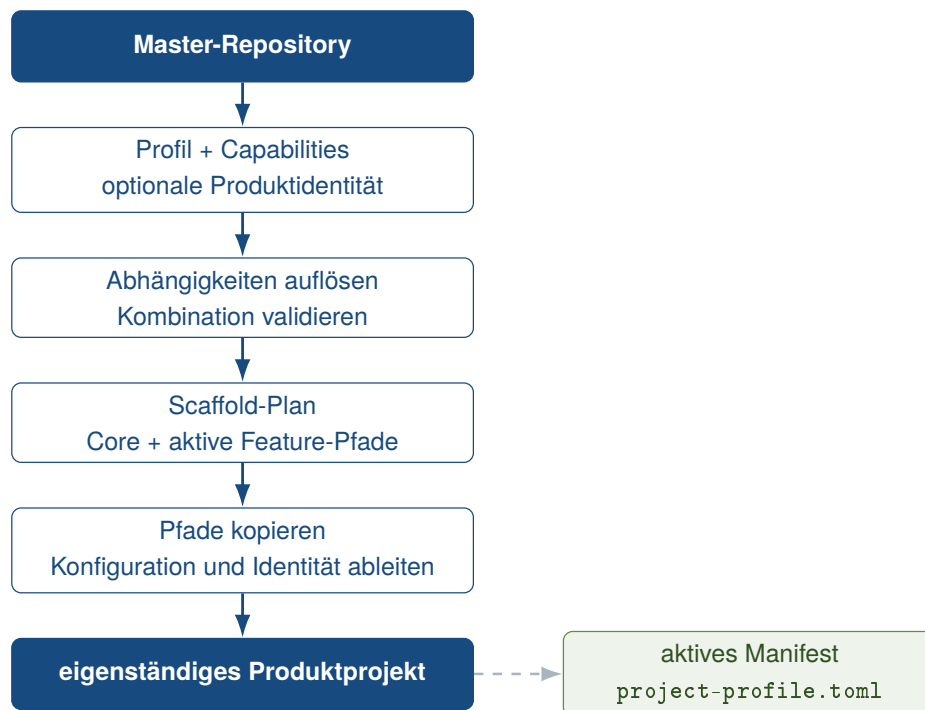


Abbildung 12: Workflow der profilgesteuerten Projektgenerierung

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026g) und kleiveist (2026l).

Der Generator löst Abhängigkeiten auf, erstellt den Scaffold-Plan und prüft Quellen und Ziel vor dem Schreiben. Er kopiert Core und aktive Features, erzeugt Manifest, Frontend-Profil und `.env.example` und passt vorhandene Produktmetadaten an. Transiente Verzeichnisse werden ausgelassen; Webprofile verlieren Tauri-Skript und -CLI. Bei angegebener Produktidentität werden nur tatsächlich vorhandene Frontend-, Backend-, Compose-, Tauri- und Cargo-Metadaten angepasst. Der Umfang bleibt damit vom gewählten Profil abhängig.

Im Master sind Ziele nur unter `.generated/` erlaubt; externe Ziele sind möglich. Das Repository selbst und nichtleere Ziele werden abgewiesen, `-dry-run` schreibt nichts. Eine Grenze bleibt: Nur der Dialog filtert nach `selectable`; direktes `-with database` wird für Backendprofile akzeptiert (kleiveist, 2026k). Diese Vorprüfungen halten Master und Ziel getrennt und verhindern teilweise erzeugte Projekte bei bekannten Konflikten. Die Ableitung ist deterministisch angelegt; der empirische Reproduzierbarkeitsnachweis bleibt dennoch auf den geprüften Fall begrenzt.

Kurz erklärt: ein Projekt erzeugen

Das Kommando `init` startet den Generator für ein neues Produktprojekt. Der Generator baut aus dem gewählten Profil ein *Scaffold*, also ein vorbereitetes Grundgerüst aus Ordnern und Dateien. Das Zielverzeichnis ist der Ort, an dem dieses Projekt angelegt werden soll. Ein *Dry Run* prüft den vorgesehenen Ablauf, ohne Dateien zu schreiben. Zur Projektidentität gehören Angaben wie Name und Kurzname, durch die das erzeugte Projekt vom Template unterschieden wird. Eine Anwendungs-ID kennzeichnet eine Tauri-Anwendung eindeutig. Ein *Reverse-Domain-Identifier* ist eine solche ID, bei der Wörter wie in `com.example.product` von der allgemeinen Internet-Domain zum konkreten Produkt angeordnet sind.

5.3 Systemprüfung und Installation

Der nur lesende `doctor` prüft profilabhängig Laufzeiten, Konfiguration, Pfade, Abhängigkeiten und Ports. Deaktivierte Schichten sind kein Fehler. Fehlendes Docker erzeugt allgemein nur eine Warnung; Container-, Datenbank- und Tauri-Voraussetzungen besitzen strengere eigene Diagnosewege (kleiveist, 2026i, 2026l).

`install` richtet nur aktive Komponenten ein: Frontend-Abhängigkeiten werden bevorzugt aus dem Lockfile installiert, Backend-Abhängigkeiten in einer virtuellen Umgebung. Datenbankpakete, Tooling-Umgebung und Chromium kommen nur bei Bedarf hinzu; für Python besteht neben `uv` ein `pip/venv`-Fallback. Eine lokale `.env` bleibt unverändert. Native Betriebssystem- und Rust-Voraussetzungen behandelt `tauri install` separat. Konkret verwendet das Frontend bei vorhandenem Lockfile `npm ci`. Ein aktives Backend erhält eine eigene Umgebung und nur die profilabhängigen Anforderungssätze. Chromium wird ausschließlich bei konfiguriertem Playwright installiert; deaktivierte Schichten werden übersprungen.

Die README fordert Git, Python ab 3.11, Node.js ab 20 und für Desktopprofile Rust Stable. Der allgemeine Doctor prüft Git und diese Versionsbereiche nicht; der Tauri-Doctor akzeptiert jede aktive Rust-Toolchain. CI richtet die festgelegten Python- und Node-Hauptversionen dagegen explizit ein. Damit weicht die lokale Durchsetzung von der Dokumentation ab (kleiveist, 2026c, 2026k). Diese Grenze betrifft die Prüfung der Voraussetzung, nicht deren dokumentierte Gültigkeit.

Kurz erklärt: Voraussetzungen und Installation

`doctor` prüft lesend, ob die für das gewählte Profil wichtigen Laufzeiten, Einstellungen, Pfade, Abhängigkeiten und Ports bereitstehen. Eine Installation richtet benötigte Abhängigkeiten ein, also zusätzliche Programme und Bibliotheken, auf denen das Projekt aufbaut. Ein *Package Manager* automatisiert das Herunterladen und Einrichten solcher Pakete. Node.js führt das Frontend-Tooling aus; `npm` verwaltet dessen Pakete, und `npm ci` installiert die im Lockfile festgelegten Stände. Python führt Backend und Tooling aus; eine virtuelle Umgebung (`venv`) trennt deren Pakete vom übrigen System, während `uv` oder ersatzweise `pip` sie installiert. Git verwaltet Quellcodeänderungen und wird zum Klonen des Repositories benötigt. Rust ist für die native Tauri-Schicht der Desktopprofile erforderlich. Chromium ist der Browser für konfigurierte Playwright-Tests und wird nur bei Bedarf eingerichtet; auch andere optionale Werkzeuge werden profilabhängig behandelt.

5.4 Entwicklungsbetrieb

Abbildung 13 zeigt den profilabhängigen Weg von Diagnose und Installation bis zu Qualitätssicherung und Auslieferung.

`run` startet Vite und nur bei aktivem Backend zusätzlich Uvicorn. Im Vordergrund beendet ein Abbruch die erfassten Prozessgruppen; im abgekoppelten Modus werden PID- und Logdaten gespeichert. `stop` beendet erfasste Prozesse und standardmäßig auch Listener auf den konfigurierten Ports; `-tracked-only` beschränkt dies auf den eigenen Zustand (kleiveist, 2026a, 2026l). Im Vordergrund werden die Ausgaben aktiver Dienste zusammengeführt. Der abgekoppelte Modus hält dagegen Prozess- und Logdaten unter `tools/.runtime`; diese Zustandsablage bildet die Grundlage für `stop`.

Der getrennte Desktopweg delegiert an `tauri dev` und entfernt bekannte serverseitige Geheimnisse aus der Kindprozessumgebung. Er unterstützt Vorder- und Hintergrundbetrieb. Die Konsole führt

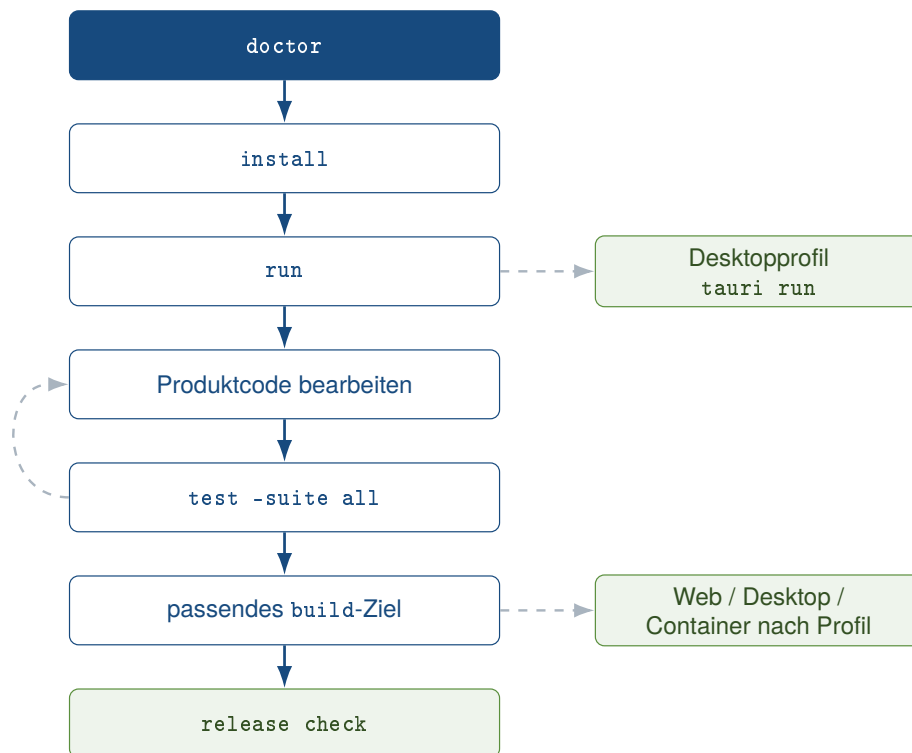


Abbildung 13: Grundstruktur des Entwicklungsworkflows

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026l) und kleiveist (2026h).

lediglich durch dieselben CLI-Workflows. Damit bleibt der native Entwicklungsweg vom allgemeinen Vite-/Uvicorn-Betrieb getrennt, ohne eine zweite Befehlslogik einzuführen.

Kurz erklärt: Programme im Entwicklungsbetrieb

Ein Entwicklungsserver macht eine noch nicht ausgelieferte Anwendung während der Arbeit lokal erreichbar. `run` startet dafür Vite und bei aktivem Backend zusätzlich Uvicorn. Vite bedient das Frontend, Uvicorn das FastAPI-Backend. Ein laufendes Programm heißt Prozess; ein Port ist dessen nummerierter Zugangspunkt für Verbindungen. Im Vordergrund bleibt die Ausgabe direkt im Terminal sichtbar und ein Abbruch beendet die erfassten Prozesse. Im Hintergrund laufen sie abgekoppelt weiter. Eine PID identifiziert dabei einen Prozess, während ein Log seine Meldungen festhält. `stop` beendet die erfassten Prozesse und je nach Option auch weitere Listener auf ihren konfigurierten Ports.

5.5 Tests und Builds

Die acht Testsuiten decken Tooling, Schema, API, Datenbank, PostgreSQL, Frontend, E2E und Tauri/Rust ab. `test -suite all` bildet den vollständigen projektbezogenen Einstieg; Berichte sind optional (kleiveist, 2026a, 2026l). Die Tooling-Suite prüft unter anderem Generator, Profile und Konfiguration; die Komponentensuiten delegieren an Pytest, Vitest, Playwright oder Cargo. Damit bleiben Testbereiche getrennt ausführbar.

Die Auswahl bleibt profilabhängig: Fehlt ein Feature, meldet die Suite `SKIP`; fehlen für ein aktives Feature Quellen oder Voraussetzungen, meldet sie `FAIL`. PostgreSQL ohne Test-URL und E2E ohne Playwright-Konfiguration werden übersprungen. Der Runner kann fehlende Backend-Umgebungen reparieren und für E2E die Entwicklungsdienste verwalten. Diese Semantik unterscheidet bewusst fehlende Features von defekten aktiven Komponenten. Ein übersprungener Test ist deshalb kein

Erfolgsnachweis.

Die Buildpfade bleiben getrennt: Web erzeugt Vite-Ausgabe und ZIP, Desktop delegiert Ziel und Bundle an Tauri, Container baut für Cloudprofile Frontend-, Backend- oder beide Images. `container validate` prüft das Compose-Modell ohne Start (kleiveist, 2026b, 2026h). Der Webpfad legt zusätzlich ein ZIP-Paket ab. Desktop-Builds bleiben an ein aktives Tauri-Feature und die jeweilige Zielplattform gebunden.

GitHub Actions verwendet dieselben öffentlichen Einstiege für Core, Profilmatrizen, PostgreSQL und Desktopziele. CI ergänzt dafür Betriebssysteme, Laufzeiten und einen temporären PostgreSQL-Dienst (kleiveist, 2026c). Lokale und externe Abläufe nutzen damit vergleichbare Einstiegspunkte, laufen aber nicht unter identischen Umgebungsbedingungen.

Kurz erklärt: Tests und Builds

Ein automatisierter Test führt vorgegebene Prüfungen ohne manuelles Durchklicken aus. Ein Unit-Test betrachtet einen kleinen, abgegrenzten Teil des Codes. Ein Integrationstest prüft, ob mehrere Teile wie Backend und Datenbank zusammenarbeiten. Ein E2E-Test verfolgt einen vollständigen Ablauf aus Sicht eines Nutzers. Eine Testsuite bündelt zusammengehörige Tests; das Template verwendet dafür Pytest bei Python, Vitest im Frontend, Playwright für E2E und Cargo bei Rust. *PASS* bedeutet bestanden, *FAIL* fehlgeschlagen und *SKIP* übersprungen; ein übersprungener Test ist kein Erfolgsnachweis. Ein Build verarbeitet den Quellcode zu einem auslieferbaren Artefakt, hier je nach Pfad etwa zur Web-Ausgabe oder zu einem Desktop-Paket. Ein ZIP bündelt Dateien in einem Archiv; ein Container-Image verpackt eine Laufzeitumgebung, aus der ein Container gestartet werden kann.

5.6 Release- und Packaging-Workflow

`VERSION` ist die zentrale Versionsquelle; Prüfung und schreibende Synchronisierung sind getrennt. `release check` erzeugt keine Artefakte. Der Befehl prüft Version, Produktidentität, Tauri-Regeln, Tag und Git-Arbeitsbaum. Inkonsistenzen, Platzhalter aus dem Template und ein unpassender oder veränderter Stand sind Fehler. Eine fehlende Signing-Konfiguration erzeugt dagegen nur eine Warnung. Nur das Master-Repository akzeptiert seine kanonische Template-Identität (kleiveist, 2026a, 2026h).

Web liefert Vite-Build und ZIP. Tauri erzeugt auf dem jeweiligen Host native Windows-, macOS- oder Linux-Bundles; weitere Wege bestehen für ein portables Windows-ZIP und einen Linux-Cross-Build. CI erzeugt nur kurzlebige, unsignierte Prüfarbeitsprodukte. Diese Pakete prüfen den technischen Packaging-Pfad. Ohne Signatur und produktspezifische Freigabe sind sie keine veröffentlichbaren Releases.

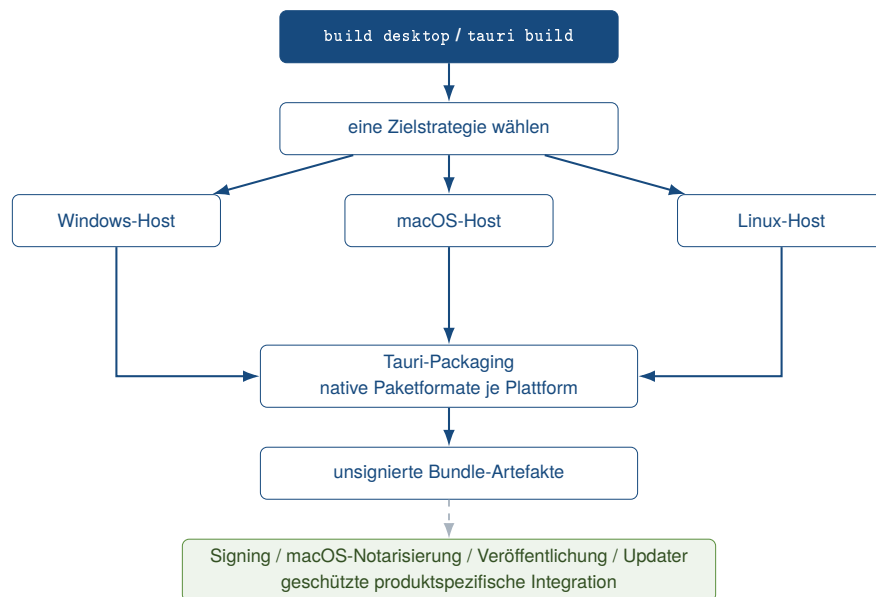


Abbildung 14: Plattformübergreifendes Modell für Desktop-Releases

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026h).

Signierung und Notarisierung sind produktspezifisch. Das gilt auch für Veröffentlichung, App-Stores und Updater. Der Release-Workflow verbindet Tests, Web- und Container-Build. Danach folgen Gate und unsignierte Desktopmatrix. Der Ablauf veröffentlicht und deployt nicht (kleiveist, 2026c, 2026h).

Für Cloudprofile prüfen die Containerbefehle Docker, Compose und Deployment-Dateien und bauen versionierte Images. Compose simuliert lokal die Produktion und führt PostgreSQL optional. Die Basis legt keinen Anbieter fest und ersetzt weder Secret-Verwaltung noch einen kontrollierten Migrationsjob (kleiveist, 2026b, 2026e). Eine Docker-/Compose-Basis ist daher nicht mit einer fertigen Architektur für AWS, Azure oder GCP gleichzusetzen.

Kurz erklärt: vom Versionsstand zum Paket

Eine Version ist eine eindeutige Bezeichnung für einen bestimmten Stand; im Template ist `VERSION` die zentrale Quelle dafür. Ein Release ist eine zur Veröffentlichung freigegebene Version, doch der untersuchte Workflow veröffentlicht selbst nichts. `release check` kontrolliert lediglich die Voraussetzungen und erzeugt keine Artefakte. Beim *Packaging* werden Programmdateien zu einem auslieferbaren Paket oder *Bundle* zusammengefasst; dieses Ergebnis ist ein Artefakt. Ein Git-Tag markiert einen bestimmten Commit mit einem Namen, häufig der Versionsnummer. Beim *Signing* bestätigt eine digitale Signatur mit einem Zertifikat die Herkunft eines Pakets; Notarisierung ist eine zusätzliche externe Prüfung, und ein Updater verteilt spätere Versionen an installierte Anwendungen. Deployment bedeutet das Bereitstellen einer Anwendung; Docker baut dafür Images, Container führen sie aus und Docker Compose beschreibt mehrere zusammengehörige Container. AWS, Azure und GCP sind mögliche Cloud-Anbieter, auf die sich diese anbieterneutrale Basis jedoch nicht automatisch festlegt.

5.7 Entwickler-Onboarding

Das Onboarding folgt dem Pfad Master klonen, `doctor`, `init`, Zielprojekt, erneut `doctor`, `install` und `run`. Produktcode, Konfiguration, Tests und Commits gehören danach in das abgeleitete Repository. Template-Wartung arbeitet dagegen im Master und prüft dort die gesamte Baseline (kleiveist, 2026g,

2026k). Diese Trennung verhindert, dass Produktentwicklung versehentlich die gemeinsame Vorlage verändert.

Grundvoraussetzungen sind Git, Python ab 3.11 und Node.js ab 20. Desktopprofile benötigen zusätzlich Rust Stable und plattformspezifische Tauri-Pakete; Docker, PostgreSQL und PyGitIndex nur für die jeweiligen optionalen Aufgaben. Optionale Werkzeuge blockieren damit nicht jedes Profil.

README und Fachdokumente trennen Einstieg, Tooling, Profile, Konfiguration, CI, Container und Release. Das vereinheitlicht die öffentlichen Setup-Pfade, belegt aber noch keine messbare Verkürzung des Onboardings (kleiveist, 2026c, 2026h, 2026i, 2026l). Die Dokumentationsstruktur ergänzt damit das technische Onboarding, ersetzt aber keine empirische Messung seiner Dauer.

Kurz erklärt: Einstieg für Entwickler

Onboarding ist der Weg, auf dem ein Entwickler das Projekt und seine Arbeitsabläufe erstmals einrichtet. Ein Repository zu klonen bedeutet, mit Git eine lokale Arbeitskopie davon anzulegen. Die Entwicklerumgebung umfasst die benötigten Programme, Abhängigkeiten und Einstellungen auf dem Arbeitsrechner. Das Master-Repository ist die gemeinsam gepflegte Vorlage; der eigentliche Produktcode gehört danach in das getrennte Produktrepository. Die Reihenfolge lautet: Master klonen → `doctor` → `init` → Zielprojekt → erneut `doctor` → `install` → `run`. Damit wird erst die Vorlage geprüft und anschließend das erzeugte Zielprojekt eingerichtet und gestartet.

6 Qualitätssicherung und Verifikation

6.1 Teststrategie

Die Verifikation ordnet jede Beobachtung einem Artefakt, einer ausgeführten Prüfung und einem commitgebundenen Ergebnis zu. Testcode allein belegt keine Ausführung, eine Workflow-Datei keinen erfolgreichen CI-Lauf. Ausführungsergebnisse und Abnahmeprotokoll wiegen daher stärker als Code, Konfiguration und Dokumentation (ISO/IEC, 2023; Runeson & Höst, 2009).

Vier Ebenen decken die Baseline ab: Tooling-Tests prüfen CLI, Profile, Generator, Konfiguration, Release und Repository; Komponententests Schema, Backend, Frontend und Tauri/Rust. Integrationstests verbinden PostgreSQL 16, Konfiguration und Alembic. Buildprüfungen erzeugen Web-, Container- oder native Artefakte (kleiveist, 2026c).

Der Umfang folgt dem aktiven Profil. *PASS* bezeichnet eine erfolgreiche, *FAIL* eine ausgeführte und gescheiterte Prüfung. *Nicht verifiziert* bedeutet fehlende Evidenz, *außerhalb des Scopes* eine bewusste Grenze; deaktivierte Gegenstände sind nicht anwendbar. Generierung und Strukturprüfung adressieren A-01 und A-03, die öffentliche Tooling-Schnittstelle A-04 und gültige sowie abgewiesene Capability-Kombinationen A-07. Abbildung 15 ordnet diese Nachweise ein.

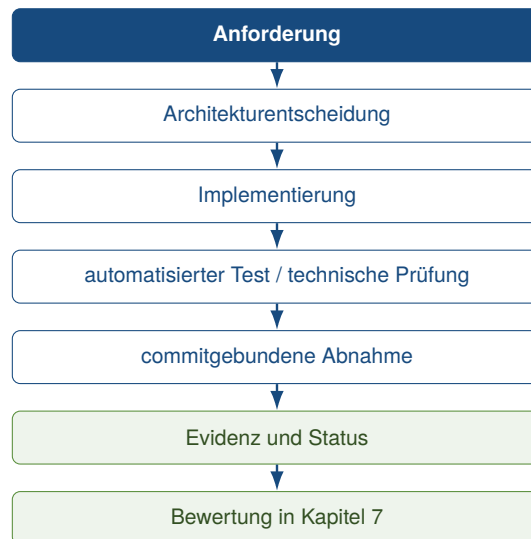


Abbildung 15: Nachweiskette der technischen Verifikation

Quelle: Eigene Darstellung.

Kurz erklärt: Qualität und Nachweise

Qualitätssicherung umfasst die Maßnahmen, mit denen Fehler gesucht und Anforderungen überprüft werden. Ein Test ist eine konkret ausgeführte Prüfung. Verifikation bedeutet hier, nachvollziehbar zu kontrollieren, ob ein erwartetes technisches Verhalten tatsächlich belegt ist. Evidenz oder ein Nachweis ist das prüfbare Ergebnis; ein Artefakt kann dabei beispielsweise ein erzeugtes Paket oder ein Protokoll sein. Ein Commit ist ein eindeutig gespeicherter Quellcodestand, weshalb ein commitgebundener Nachweis nur für genau diesen Stand gilt. *PASS* steht für bestanden, *FAIL* für ausgeführt und fehlgeschlagen und *SKIP* für nicht ausgeführt oder nicht anwendbar. Fehlende Evidenz heißt deshalb *nicht verifiziert* und ist weder ein Erfolg noch automatisch ein Fehler.

6.2 Continuous Integration

Core CI, Profile Matrix, PostgreSQL Integration und Desktop CI prüfen getrennt Baseline, fünf Profile, PostgreSQL-Kombinationen und native Pakete. Release Validation läuft nur manuell oder für Versions-tags. Alle Workflows besitzen Leserechte und verwenden `tools/control.py` (kleiveist, 2026c).

Die am 9. August 2026 abgefragten Läufe vom 7. August gehören zu HEAD `a4487ac` und sind nicht vollständig grün. Core CI war für Backend, Schema, Frontend, Web-Build, Compose und beide Images erfolgreich; Tooling, Profile und Konfiguration scheiterten. In der Profilmatrix bestanden beide Webprofile, die drei Desktopprofile scheiterten beim Projekttest. Die PostgreSQL-Integration war für Master und Web-cloud erfolgreich, nicht für die beiden Desktopkombinationen. Desktop CI paketierte unter Linux und macOS; Windows scheiterte bei der Frontend-Installation (GitHub, 2026a, 2026c, 2026d, 2026e).

Lokal bestanden am 9. August 2026 auf demselben HEAD unter Python 3.13.14 alle 194 Tooling-Tests sowie Schema-, API- und SQLAlchemy-Tests. Das ersetzt weder die roten externen Jobs noch die mangels Node.js, Rust und Docker nicht ausgeführten Pfade. Release Validation wurde für diesen Commit nicht ausgeführt.

Kurz erklärt: Continuous Integration

Continuous Integration (CI) bedeutet, dass Prüfungen nach Quellcodeänderungen automatisch auf einem separaten System ausgeführt werden. Im untersuchten Projekt übernimmt GitHub Actions diesen Dienst. Ein *Workflow* beschreibt einen solchen automatischen Ablauf. Darin ist ein *Job* eine einzelne Arbeitsgruppe von Schritten, die auf einem *Runner*, also dem bereitgestellten Rechner, läuft. Eine *Pipeline* ist die gesamte Kette dieser automatischen Prüfungen. Ein Commit bezeichnet einen festen Codezustand; *HEAD* zeigt auf den Commit, der für die betrachtete Arbeitskopie oder den Zweig gerade maßgeblich ist. Eine grüne CI hat alle verlangten Jobs bestanden, eine rote CI enthält mindestens einen Fehlschlag. Die im Haupttext ausgewerteten Läufe für Commit a4487ac sind nicht vollständig grün, auch wenn bestimmte lokale Prüfungen bestanden.

6.3 Abnahme und technische Verifikation

Das finale Abnahmeprotokoll bewertet Commit 1cdfb6e als *PASS WITH OPEN EXTERNAL ITEMS* und *READY FOR CASE STUDY*. Dies gilt für die Fallstudien-Baseline, nicht für Produktionsreife oder externe Deployments. Auf diesem Commit bestanden 189 Tooling-Tests (kleiveist, 2026j).

Die Abnahme generierte und installierte alle fünf Profile, führte profilabhängige Diagnosen, Tests und Web-Builds aus und paketierte die drei Desktopprofile als Linux-DEB. Mit bereitgestelltem Compose bestand die strikte Validierung. Die drei Backendprofile bestanden zusätzlich mit PostgreSQL samt Verbindung, Migrationen, Tests und Build; Desktopvarianten wurden erneut paketiert. Die beiden inkompatiblen PostgreSQL-Kombinationen wurden vor der Zielerzeugung erwartungsgemäß abgewiesen (kleiveist, 2026j).

Abbildung 16 und Tabelle 5 trennen lokale Abnahme und aktuelle CI. Die neueren Fehler entwerten die commitgebundene Abnahme nicht, verhindern aber ihre Übertragung auf a4487ac als vollständig grünen Remote-Nachweis.

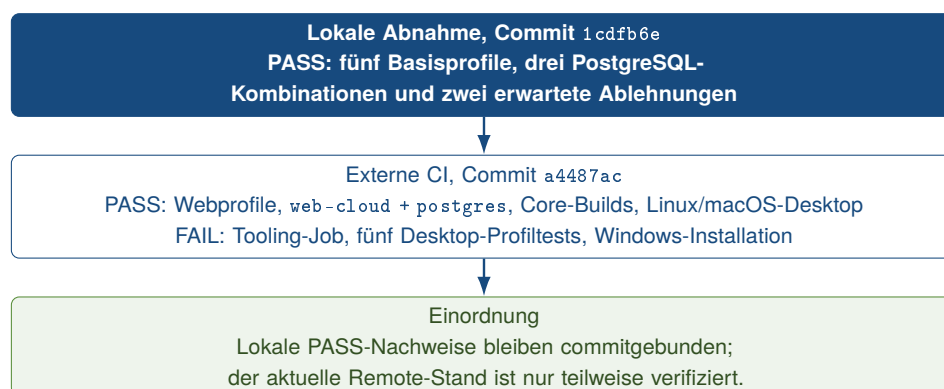


Abbildung 16: Profilverifikation nach Evidenzstand und Commit

Quelle: Eigene Darstellung auf Grundlage von kleiveist (2026j) und den aktuellen GitHub-Actions-Läufen.

Tabelle 5: Profilbezogene technische Verifikation

Profil / Kombination	Gen.	Install / Doctor	Tests	Web	Desktop	PG / Migr.	CI auf a4487ac
web-only	P	P	P	P	–	–	P
web-cloud	P	H	P	P	–	–	P
desktop-local	P	P	P	P	P (Linux DEB)	–	F: Profilttest
desktop-cloud	P	H	P	P	P (Linux DEB)	–	F: Profilttest
full-platform	P	H	P	P	P (Linux DEB)	–	F: Profilttest
web-cloud + postgres	P	P	P	P	–	P	P
desktop-cloud + postgres	P	P	P	P	P (Linux DEB)	P	F: Profilttest
full-platform + postgres	P	P	P	P	P (Linux DEB)	P	F: Profilttest

Legende: P = PASS; H = PASS WITH NOTE; F = ausgeführte Prüfung fehlgeschlagen; – = nicht anwendbar. Die sechs mittleren Ergebnisspalten stammen aus der lokalen Abnahme auf 1cdfb6e; die letzte Spalte gibt den profilbezogenen GitHub-Actions-Run auf a4487ac wieder. web-only + postgres und desktop-local + postgres wurden in der lokalen Abnahme vor der Zielerzeugung korrekt abgewiesen (P). Quelle: kleiveist (2026j) sowie GitHub (2026d, 2026e).

Kurz erklärt: technische Abnahme

Eine technische Abnahme prüft anhand festgelegter Schritte, ob eine Softwarebasis die vorgesehenen technischen Anforderungen erfüllt. Ein lokaler Nachweis entsteht auf einem Arbeitsrechner; ein Remote-Nachweis stammt etwa von einem externen CI-System. Commitgebunden bedeutet, dass das Ergebnis nur für den dabei geprüften Quellcodezustand gilt. Die Abnahme für 1cdfb6e bestand mit offenen externen Punkten und bestätigt die Eignung als Fallstudien-Baseline. Sie macht spätere Fehler auf a4487ac nicht zu bestandenen Remote-Prüfungen. Produktionsreife würde zusätzlich belegen, dass ein Produkt sicher und zuverlässig im echten Betrieb eingesetzt werden kann. Diese Produktionsreife oder ein externes Deployment wird durch die Abnahme ausdrücklich nicht bestätigt.

6.4 Reproduzierbarkeit

Zwei Generierungen von desktop-cloud + postgres waren auf 1cdfb6e bei identischen Eingaben bytgleich. `init -dry-run` schrieb nichts; nichtleere Ziele und inkompatible Capabilities wurden vor dem Schreiben abgewiesen. Der Nachweis gilt nur für diese Kombination und Umgebung (kleiveist, 2026j; Lamb & Zacchiroli, 2022).

Frontend-, Cargo- und drei Python-Lockfiles fixieren Abhängigkeiten; npm und Cargo verwenden gesperrte Stände, die Container-Baseline zusätzlich Hashes. Die hashgebundene Installation wurde geprüft (kleiveist, 2026b, 2026c). Betriebssystem, Toolchain, Registries, Netzwerk und Plattform-SDKs begrenzen dennoch die Reproduktion; native Ergebnisse bleiben plattformspezifisch.

Kurz erklärt: reproduzierbare Ergebnisse

Ein reproduzierbarer Build soll bei gleichen Eingaben wieder dasselbe Ergebnis erzeugen. Für diese Fallstudie wurde jedoch nur gezeigt, dass zwei Generierungen von `desktop-cloud + postgres` auf `1cdfb6e` in derselben Umgebung bytgleich waren. Eine festgelegte Dependency-Version bestimmt genau, welcher Stand einer benötigten Bibliothek verwendet wird. Ein Lockfile schreibt solche Versionen fest, statt beim nächsten Lauf beliebige neuere Pakete zu wählen. Ein Hash ist ein kurzer digitaler Fingerabdruck, mit dem sich ein bestimmter Inhalt sehr zuverlässig prüfen lässt. Die Toolchain umfasst die eingesetzten Laufzeiten, Compiler und Buildwerkzeuge. Unterschiede bei Betriebssystem, Toolchain, Netzwerk oder Plattform-SDKs können die Wiederholung trotzdem beeinflussen.

6.5 Security und Konfigurationsgrenzen

Tooling-Tests belegen die Konfigurationspriorität sowie die Validierung von Umgebungen, Ports und CORS-Ursprüngen. Sie prüfen auch die profilabhängige `.env.example` und `DATABASE_URL` nur für PostgreSQL. Lokal bestanden diese Tests, während der externe Tooling-Job auf demselben Commit scheiterte (GitHub, 2026a; kleiveist, 2026i).

Nur `VITE_API_BASE_URL` darf in das Frontend gelangen. Tests verwerfen öffentliche Secrets, entfernen `DATABASE_URL` aus Frontend und Tauri und maskieren Datenbankpasswörter in Diagnosen. Ohne PostgreSQL entsteht keine Datenbankvariable. Weitere Prüfungen decken Health, Readiness, Fehlerantworten, datenbankunabhängige Liveness sowie Tauri-CSP und Capability-Metadaten ab (kleiveist, 2026h, 2026j).

Verifiziert sind damit nur Konfigurations-, Secret- und Desktop-Grenzen der Baseline. Authentifizierung, Berechtigungen, produktive Credentials, TLS und providerspezifische Härtung bleiben produktspezifisch; eine vollständige Sicherheitsbewertung ist nicht belegt.

Kurz erklärt: Sicherheitsgrenzen

Security bezeichnet den Schutz einer Anwendung, ihrer Daten und ihrer Zugänge vor unerlaubter Nutzung. Ein *Secret* ist ein vertraulicher Wert; *Credentials* sind Zugangsdaten, mit denen sich jemand oder ein Dienst ausweist. CORS legt fest, von welchen Web-Ursprüngen ein Browser Anfragen zulassen darf; eine CSP begrenzt, welche Inhalte und Aktionen eine Anwendung laden oder ausführen darf. TLS verschlüsselt die Übertragung über ein Netzwerk. Authentifizierung beantwortet die Frage: *Wer bist du?* Autorisierung beantwortet die Frage: *Was darfst du?* RBAC setzt solche Rechte über zugewiesene Rollen um. Verifiziert sind hier nur bestimmte Konfigurations-, Secret- und Desktop-Grenzen der Baseline, während Authentifizierung, Autorisierung, produktive Zugangsdaten, TLS und eine vollständige Sicherheitsbewertung Aufgabe des konkreten Produkts bleiben.

6.6 Extern verbleibende Prüfungen

Tabelle 6 trennt fehlgeschlagene Prüfungen, fehlende Evidenz und bewusste Produktgrenzen. Rote CI-Jobs sind reale Fehler; ein nicht gestarteter Compose-Verbund ist dagegen nicht verifiziert.

Der aktuelle Core-Job belegt Compose-Validierung und beide Master-Images, aber weder `docker compose up` noch produktives Deployment. Linux- und macOS-Pakete wurden auf `a4487ac` unsigniert gebaut; Windows nicht. Branch Protection war mangels API-Berechtigung nicht lesbar. Signing, Notarisierung, Updater, produktive Zugangsdaten und Authentifizierung/RBAC benötigen ein Produkt und

externe Infrastruktur (GitHub, 2026a, 2026c; kleiveist, 2026h).

Tabelle 6: Offene und externe Verifikationspunkte auf a4487ac

Prüfpunkt	Status	Grund	erforderlicher Nachweis
Tooling- und Desktop-Profil-CI	FAIL	aktuelle Jobs enden in ausgeführten Testschritten mit Exitcode 1	erfolgreiche Wiederholung auf dem korrigierten Commit
Windows-Paket	FAIL	Frontend-Installation im nativen Windows-Job fehlgeschlagen	erfolgreicher nativer Build und Artefakt-Upload
Release Validation	nicht verifiziert	kein manueller oder Tag-Run vorhanden	erfolgreicher, nicht publizierender Run auf festem Commit
Compose-Start	nicht verifiziert	nur Modell und Master-Images in CI geprüft	<code>docker compose up</code> , Health-/Ready-Nachweis und kontrollierter Abbau
Branch Protection	nicht verifiziert	öffentliche API verlangt Authentifizierung	autorisierter Nachweis der Required Checks für <code>main</code>
Signing, Notarisierung, Updater	außerhalb des Scopes	produktspezifische Identität und geschützte Credentials fehlen bewusst	geschützte native Release-Pipeline je Zielplattform
Produktivbetrieb und Auth/RBAC	außerhalb des Scopes	Cloud, Geschäftsmodell und Sicherheitsmodell sind produktspezifisch	produktbezogene Abnahme einschließlich Credentials und Deployment

Quelle: Eigene Zusammenstellung aus kleiveist (2026h, 2026j) und den aktuellen GitHub-Actions-Läufen.

Kurz erklärt: was extern noch zu prüfen ist

Externe Prüfungen brauchen Dienste, Berechtigungen oder Infrastruktur außerhalb der lokalen Vorlage. *Branch Protection* sind Regeln, die einen Codezweig beispielsweise nur nach bestandenen Prüfungen ändern lassen. Deployment ist das kontrollierte Bereitstellen einer Anwendung in ihrer Zielumgebung. Signing versteht ein Paket mit einer prüfbaren digitalen Signatur; bei der Notarisierung kontrolliert zusätzlich eine externe Plattformstelle das Paket. Ein Updater verteilt neue Versionen an bereits installierte Anwendungen. Produktionsbetrieb umfasst den dauerhaften, überwachten Einsatz für echte Nutzer. Für die betrachtete Baseline sind diese Punkte nicht vollständig nachgewiesen: Branch Protection war nicht lesbar, die erzeugten Pakete waren unsigniert und ein produktives Deployment wurde nicht ausgeführt.

7 Bewertung des Technologie-Templates

7.1 Produktivitätswirkung

Die Bewertung stellt Anforderungen, implementierte Mechanismen und Evidenz gegenüber. Kriterien sind Wiederverwendung, Standardisierung, Variabilität, Wartbarkeit, Testbarkeit, Reproduzierbarkeit, Entwicklerfreundlichkeit, Erweiterbarkeit und Plattformabdeckung. Komplexität, Betriebsreife und Dokumentierbarkeit ergänzen die Einordnung. Das projektinterne Abnahmeprotokoll liefert Projektnachweise, aber keine unabhängige Replikation (kleiveist, 2026j). Damit bleiben beobachtete Mechanismen, ausgeführte Nachweise und abgeleitete Wirkungen getrennt. Automatisierung gilt nicht allein deshalb als messbarer Produktivitätsgewinn.

Produktivität umfasst hier den Aufwand bis zum lauffähigen Ausgangsprojekt, wiederkehrende Einrichtungsentscheidungen, den Entwicklungsfluss und die spätere Pflege. Eine einzelne Aktivitätskennzahl bildet sie nicht angemessen ab (Forsgren et al., 2021). Konzeptionell lässt sich die Wirkung daher als

$$\text{Produktivitätsnutzen} = \text{vermiedener Projektaufwand} - \text{Template-Pflegeaufwand} - \text{zusätzliche}$$

auffassen. Mangels Zeit- und Aufwandsmessungen ist sie nicht numerisch auswertbar. Die Beziehung zeigt zugleich, dass zentrale Pflege und zusätzliche Komplexität den vermiedenen Projektaufwand mindern können.

Beim Projektstart bündelt `control.py` Profilwahl, Generierung, Systemprüfung und Installation. Die Abnahme belegt diese Schritte für alle fünf Profile sowie die Ablehnung inkompatibler PostgreSQL-Auswahlen (kleiveist, 2026j). Die gemischten CI-Ergebnisse des späteren HEAD begrenzen die Übertragbarkeit. Weniger manuelle Einrichtung ist daher plausibel, eine konkrete Zeitersparnis aber nicht belegt.

In der Entwicklung vereinheitlichen profilabhängige Einstiege Start, Test und Build. Gemeinsames Frontend, Konfigurationsvertrag und klare Verantwortungen fördern Wiederverwendung; Fachwerkzeuge und Plattformvoraussetzungen bleiben. Zentrale Änderungen nützen künftigen Projekten, erfordern aber gemeinsame Pflege von Profilen, Generator, Tests und Dokumentation. Bereits generierte Projekte werden nicht automatisch aktualisiert. Der Nutzen verteilt sich somit über Projektstart, laufende Entwicklung und spätere Baseline-Pflege. Er ist besonders plausibel, wenn mehrere technisch verwandte Vorhaben dieselben Grundlagen verwenden.

Quantitative Evidenz fehlt. Eine spätere kontrollierte Evaluation müsste vergleichbare Projekte mit und ohne Template anhand von Start- und Onboarding-Zeit, Setup-Fehlern, fehlgeschlagenen Builds, Wiederverwendungsanteil und Pflegezeit untersuchen. Zusätzlich wären die Zeit bis zur ersten Fachfunktion und die Zahl manueller Einrichtungsentscheidungen zu erfassen. Erst solche Vergleichsdaten erlauben eine Aussage über die Größe des Effekts.

Kurz erklärt: Produktivität und Messung

Entwicklerproduktivität beschreibt, wie gut ein Entwicklungsteam seine Zeit in nutzbare Ergebnisse umsetzt. Ein Template kann wiederkehrende Einrichtung vereinfachen und dadurch ein Produktivitätspotenzial schaffen.

Ein solches Potenzial ist zunächst nur ein möglicher Nutzen und noch kein gemessener Zeitgewinn.

Für eine quantitative Messung müssten vergleichbare Zahlen erhoben werden, zum Beispiel zur Einrichtungszeit, zu Fehlern und zum Pflegeaufwand.

Da solche Vergleichsdaten fehlen, bewertet die Fallstudie die Wirkung als plausibel, aber nicht als zahlenmäßig belegt.

7.2 Stärken

Die gemeinsame Baseline verbindet zentrale Pflege mit klaren Grenzen zwischen Frontend, Backend, Desktop, Persistenz und Tooling. Das geteilte Frontend adressiert A-02 und A-05, ohne jedes Profil mit allen Komponenten zu belasten. Diese Trennung unterstützt eigene Tests und produktspezifische Erweiterungen, ohne die gemeinsame Präsentationsschicht zu duplizieren.

Fünf Profile bilden wiederkehrende Plattformzuschnitte ab. Abhängigkeiten verhindern PostgreSQL ohne Backend. Die Abnahme aller Basisprofile sowie der zulässigen und unzulässigen PostgreSQL-Kombinationen stützt dieses Modell für den begrenzten aktuellen Katalog, nicht für beliebig viele künftige Capabilities (kleiveist, 2026j).

`control.py` vereinheitlicht den Einstieg, während Fachwerkzeuge sichtbar bleiben. Manifest, Konfigurationsvertrag, Tests und Buildpfade erhöhen die Nachvollziehbarkeit. Die byte-identische Generierung zweier `desktop-cloud` + `postgres`-Projekte liefert dafür einen konkreten, aber auf eine Kombination und Umgebung begrenzten Nachweis (kleiveist, 2026j). Profilabhängiges Überspringen nicht vorhandener Schichten hält die gemeinsame Befehlsoberfläche dabei an den tatsächlichen Projektumfang gebunden.

Tabelle 7 stellt Stärken und Geltungsgrenzen gegenüber.

Tabelle 7: Evidenzbasierte Gegenüberstellung von Stärken und Grenzen

Bereich	Stärke und Evidenz	Grenze und Evidenz	Bedeutung
Architektur	Getrennte Frontend-, Backend-, Tauri- und Persistenzgrenzen; gemeinsames Frontend.	Shared Schemas werden nicht automatisch von Frontend und Backend konsumiert.	Wiederverwendung bei klarer Zuständigkeit; Vertragsänderungen bleiben manuell.
Variabilität	Fünf deklarative Profile; Abhängigkeiten und drei zulässige PostgreSQL-Kombinationen sind abgenommen.	Nur eine auswählbare Capability; direkte CLI trennt <code>selectable</code> nicht strikt.	Kontrollierte Auswahl im aktuellen Katalog, begrenzte Aussage für Erweiterungen.
Tooling	Gemeinsamer profilabhängiger Einstieg für Diagnose, Installation, Start, Test und Build.	Mehrere Fachwerkzeuge und native Voraussetzungen bleiben erforderlich.	Weniger unterschiedliche Bedienpfade, aber kein technologieunabhängiger Workflow.
Qualität	Tests, Profilmatrix, Buildpfade und Release-Gate sind im Abnahmeprotokoll erfasst.	Aktuelle Tooling-, Desktopprofil- und Windows-Jobs schlagen fehl; Branch Protection ist nicht nachgewiesen.	Gute lokale Prüfbasis, aber kein durchgehend grüner Remote-Nachweis.
Reproduzierbarkeit	Zwei gleich konfigurierte Projekte waren im dokumentierten Versuch byte-identisch.	Nachweis nur für eine Kombination und eine Untersuchungsumgebung.	Konkrete Evidenz ohne Anspruch auf vollständige plattformübergreifende Reproduzierbarkeit.
Betrieb und Plattformen	Web- und Image-Builds, PostgreSQL-Integration sowie unsignierte Linux-/macOS-Pakete sind dokumentiert.	Compose-Start, produktiver Cloudbetrieb, Windows-Paket, Signing und Notarisierung bleiben offen.	Template-Reife ist nicht mit Produktionsreife gleichzusetzen.
Wartung	Core, Profile, Tooling, Tests und Dokumentation liegen in einer Baseline.	Änderungen können mehrere Profile betreffen; Produktprojekte erhalten keine automatischen Updates.	Zentrale Pflege wird gebündelt, erfordert aber eine breite Regression und Transferprozesse.

Quelle: Eigene Bewertung auf Grundlage der Kapitel 3–6 sowie des Abnahmeprotokolls und der aktuellen CI-Läufe (GitHub, 2026a, 2026c, 2026d, 2026e; kleiveist, 2026j).

7.3 Schwächen und Grenzen

Zu unterscheiden sind technische Schwächen, bewusste Scope-Grenzen und nicht verifizierte Bereiche. Nur die erste Gruppe bezeichnet unmittelbar einen Nachteil der Umsetzung. Scope-Grenzen erzeugen zwar Anpassungsbedarf, sind aber keine fehlgeschlagene Anforderung. Fehlende Verifikation erlaubt noch kein positives oder negatives technisches Urteil.

Die Bündelung im Master erhöht die interne Komplexität: Änderungen an Core, Generator oder Katalog erfordern Regressionstests über mehrere Profile. Außerdem nutzen Frontend und Backend die gemeinsamen JSON-Schemata nicht zur Laufzeit; Vertragsänderungen bleiben manuell. Die direkte CLI-Auswahl trennt `optional` und `selectable` schwächer als der Dialog. Beide Abweichungen erzeugen Wartungs- oder Fehlbedienungsrisiken. Auf dem untersuchten HEAD kommen widersprüchliche Nachweise hinzu: Webprofile und der lokale Tooling-Lauf bestanden, externe Tooling-, Desktopprofil- und Windows-Prüfungen scheiterten. Dieser Befund stellt die frühere commitgebundene Abnahme nicht rückwirkend infrage, verhindert aber ein vollständig grünes Urteil für den neueren Stand.

Bewusst ausgeschlossen sind Backend in `web-only`, Cloud-API in `desktop-local` und PostgreSQL ohne Backend. Auch Fachlogik, Authentifizierung, Rollenmodelle, verpflichtende Persistenz, Cloud-Provider und native Fachkommandos bleiben produktspezifisch. Der daraus folgende Anpassungsaufwand ist keine Abweichung vom Scope.

Nicht abschließend verifiziert sind Compose-Start, produktive Bereitstellung, Release Validation und Branch Protection. Linux und macOS wurden unsigned paketiert, Windows scheiterte. Signing, Notarisierung, Veröffentlichung und produktiver Cloudbetrieb bleiben produkt- oder infrastrukturspezifisch (GitHub, 2026a, 2026c; kleiveist, 2026j). Die Paketergebnisse belegen daher weder signierte Releases noch produktiven Cloudbetrieb. Bewertet wird die Template-Baseline, nicht die Produktionsreife eines Produkts. Die verbleibenden Risiken betreffen damit vor allem Betrieb, Plattformen und Freigabeprozesse.

Kurz erklärt: Schwäche, Grenze und Nachweis

Eine technische Schwäche ist ein tatsächlicher Nachteil der vorhandenen Umsetzung.

Eine Scope-Grenze legt dagegen bewusst fest, was das Template nicht abdecken soll.

Fehlende Verifikation bedeutet, dass eine Funktion noch nicht ausreichend geprüft oder nachgewiesen wurde; sie ist deshalb nicht automatisch fehlerhaft.

Eine Regression ist ein Fehler, der durch eine Änderung in einem zuvor funktionierenden Bereich entsteht.

7.4 Wartungsaufwand

Die Single-Repository-Strategie bündelt Core, Features, Profile, Generator, Tests und Dokumentation. Das vermeidet parallele Baseline-Änderungen und nützt künftigen Projekten. Bereits generierte Repositories müssen Korrekturen jedoch eigenständig übernehmen. Die Source of Truth reduziert somit die Zahl zu pflegender Ausgangsstände, nicht aber den Aufwand für die Migration bestehender Produkte.

Die zentrale Pflege umfasst mehrere Sprachen, Dependency-Manager, Toolchains, Plattformen und Releasezyklen. Diese Vielfalt folgt aus den Web-, Backend-, Desktop- und Deploymentzielen, erhöht aber Kompetenzbedarf, Testbreite und Umgebungsabhängigkeit. Native Builds und externe Dienste sind dabei wartungsintensiver als ein rein webbasierter, technologiehomogener Starter.

Fünf Profile, sechs Features und eine auswählbare Capability begrenzen derzeit den Kombinationsraum; die Abnahme deckt fünf Basispfade und drei zulässige PostgreSQL-Erweiterungen ab (kleiveist, 2026j). Weitere Capabilities erweitern Abhängigkeiten und Testmatrix. Eine kombinatorische Explosion ist nicht nachgewiesen, ein wachsender Pflegebedarf aber plausibel. Katalog, Resolver, Konfigurationsvertrag, Testmatrix und Dokumentation müssen bei solchen Erweiterungen gemeinsam konsistent bleiben.

Die zentrale Wartung ist damit aufwendiger als bei einem kleinen homogenen Starter, kann aber wiederholte Basispflege bündeln. Ihre Wirtschaftlichkeit hängt von Nutzungshäufigkeit, Projektähnlichkeit und Pflegezeit ab; Kostendaten fehlen.

Kurz erklärt: Zentrale Wartung

Eine Source of Truth ist die eine verbindliche Quelle, von der die gemeinsame technische Ausgangsbasis abgeleitet wird.

Der Wartungsaufwand umfasst alle Arbeiten, die diese Basis aktuell, konsistent und funktionsfähig halten.

Ein Regressionstest prüft, ob eine Änderung bereits funktionierende Teile unbeabsichtigt beschädigt hat.

Eine Toolchain ist die Gesamtheit der benötigten Entwicklungs-, Test- und Buildwerkzeuge.

Im untersuchten Template vermeidet die zentrale Quelle parallele Baseline-Pflege, doch mehrere Toolchains sowie bereits erzeugte Projekte verursachen weiterhin Wartungsarbeit.

7.5 Vergleich mit alternativen Template-Strategien

Verglichen werden Architekturstrategien anhand von Wiederverwendung, Konsistenz, Variabilität, Einstiegskomplexität, Pflege, Testbarkeit und Updateübernahme (Clements & Northrop, 2001; Krueger, 1992). Die qualitativen Stufen bilden kein Gesamtranking; hohe Einstiegskomplexität ist beispielsweise nachteilig.

Separate Template-Repositories bleiben einzeln übersichtlich, können aber divergieren. Ein *Copy-Paste-Starter* minimiert die Vorlagenlogik, liefert jedoch keinen Rückkanal für spätere Verbesserungen. GitHub-Templates erzeugen entsprechend eine eigene, nicht verwandte Historie (GitHub, 2026b). Separate Repositories gewichten damit organisatorische Isolation höher als die zentrale Konsistenz gemeinsamer Bausteine.

Framework-spezifische Starter wie `create-vite` bieten einen einfachen Einstieg in einen engen Ausschnitt. Backend, Desktop, Persistenz und Gesamtworkflow müssen bei Bedarf separat ergänzt werden (VoidZero Inc. and Vite Contributors, 2026). Für ein kleines homogenes Frontend kann diese geringere Einstiegskomplexität angemessener sein als der untersuchte Full-Stack-Ansatz.

Ein *monolithisches Universal-Template* vereinfacht die Auswahl, belastet kleine Projekte aber mit ungenutzten Komponenten. Das untersuchte *Single-Repository mit Profilen und Capabilities* kombiniert zentrale Pflege mit ausgewählten Pfaden. Dafür steigen Master-, Resolver- und Matrixkomplexität; Updates erreichen generierte Projekte nicht automatisch. Es verbindet somit zentrale Variabilität vor der Generierung mit der Eigenständigkeit eines Copy-Starters danach.

Tabelle 8: Qualitativer Vergleich von Template-Strategien

Strategie	fortlaufende Wiederverwendung	Konsistenz	Variabilität	Einstiegs-komplexität	Pflege-bündelung	Updates bestehender Projekte
Separate Template-Repositories	mittel	mittel	mittel	niedrig	niedrig	je Vorlage beziehungsweise Projekt manuell
Copy-Paste-Starter	niedrig nach Kopie	niedrig	hoch	niedrig	niedrig	keine automatische Übernahme
Framework-spezifischer Starter	hoch im engen Scope	hoch im engen Scope	niedrig bis mittel	niedrig	hoch im Starter	meist manuell nach Generierung
Monolithisches Universal-Template	mittel	hoch	niedrig	hoch	hoch	abhängig vom gewählten Ableitungsweg
Single-Repository mit Profilen	hoch im definierten Scope	hoch	hoch, kontrolliert	mittel	hoch	im Master zentral, in abgeleiteten Projekten manuell

Quelle: Eigene qualitative Gegenüberstellung auf Grundlage von Krueger (1992), Clements & Northrop (2001), GitHub (2026b) und VoidZero Inc. and Vite Contributors (2026).

Der profilbasierte Ansatz passt vor allem zu wiederkehrenden, technisch verwandten Vorhaben. Kleine Einzelprojekte können von Framework-Startern, organisatorisch getrennte Produktlinien von separaten Repositories profitieren. Es gibt keinen universellen Sieger, sondern unterschiedliche Trade-offs.

Kurz erklärt: Template-Strategien

Ein Trade-off ist eine Abwägung, bei der ein Vorteil mit einem Nachteil verbunden ist.

Ein Framework-Starter bietet einen einfachen Einstieg für einen eng begrenzten Technikbereich, etwa nur für das Frontend.

Ein monolithisches Template enthält alle vorgesehenen Bausteine gemeinsam, auch wenn ein kleines Projekt einige davon nicht benötigt.

Bei einem Copy-Paste-Starter wird die Ausgangsbasis einmal kopiert; spätere Verbesserungen der Vorlage gelangen nicht automatisch in das Projekt.

Die Profilstrategie der Fallstudie wählt Bausteine vor der Generierung aus, nimmt dafür aber zusätzliche Pflege- und Auswahlkomplexität in Kauf.

7.6 Gesamtbewertung

Tabelle 9 unterscheidet weitgehend und teilweise adressierte Kriterien von nicht abschließend nachgewiesenen. Diese Stufen sind qualitativ und messen weder Qualität noch Produktivität numerisch.

Tabelle 9: Zusammengeführte Bewertung des Technologie-Templates

Kriterium	Evidenzbasis	Bewertung	wesentliche Einschränkung
Wiederverwendung und Standardisierung	Gemeinsamer Kern, fünf Profile, Generator und einheitliche Befehlsoberfläche	weitgehend adressiert	Gilt für den definierten Stack; keine automatische Aktualisierung abgeleiteter Projekte.
Variabilität und Erweiterbarkeit	Profil-Presets, Capability-Abhängigkeiten und geprüfte PostgreSQL-Kombinationen	weitgehend adressiert	Kleiner aktueller Katalog; Erweiterungen vergrößern Pflege- und Testumfang.
Testbarkeit und Reproduzierbarkeit	Dokumentierte Profil-, Integrations- und Buildabnahme sowie ein byte-identischer Generierungsvergleich	teilweise adressiert	Aktuelle Remote-Jobs teilweise fehlgeschlagen; Reproduzierbarkeitsversuch nur für eine Kombination.
Entwicklerfreundlichkeit und Dokumentierbarkeit	<code>control.py</code> , Diagnosewege, versionierte Manifeste und gegliederte Dokumentation	teilweise adressiert	Onboarding und Bedienaufwand nicht empirisch gemessen; mehrere Toolchains bleiben sichtbar.
Wartbarkeit	Eine zentrale Baseline mit gemeinsamen Tests und Dokumentationsregeln	teilweise adressiert	Technologievielfalt, Variantenregression und manuelle Schnittstellensynchronisierung.
Plattformabdeckung und Betriebsreife	Web-, Backend-, PostgreSQL-, Container-Image-, Linux- und macOS-Nachweise; providerneutrale Grenze	nicht abschließend nachgewiesen	Windows-Paket, Compose-Start, Signing, Notarisierung und produktives Deployment nicht belegt.

Quelle: Eigene Bewertung auf Grundlage der vorangehenden Kapitel, kleiveist (2026j) und der in Abschnitt 6.2 ausgewerteten CI-Läufe.

Die Baseline ist für die definierten Profile technisch tragfähig. Kern, Presets, Abhängigkeiten und Tooling unterstützen Wiederverwendung, Standardisierung und Flexibilität. Die Abnahme stützt Generierung, Installation, Tests, Builds, PostgreSQL-Kombinationen und Linux-Packaging (kleiveist, 2026j). Neuere externe Läufe ergänzen Web-, Container-, Linux- und macOS-Nachweise, enthalten aber Fehler in Tooling-, Desktopprofil- und Windows-Jobs. Die Evidenz ist damit belastbar, aber nach Commit und Plattform unterschiedlich weitreichend.

Grenzen sind zentrale Pflegekomplexität, manuell synchronisierte Verträge und fehlende Updates generierter Projekte. Windows-Paketierung, signierte Releases, Notarisierung und produktiver Cloudbe-

trieb sind nicht nachgewiesen. Bewusst ausgelassene Provider- und Fachlogik ist dagegen kein Fehler. Tabelle 9 macht den Unterschied zwischen adressierter Anforderung und offener Nachweisbreite sichtbar.

Automatisierung und gemeinsame Workflows schaffen plausibles, aber nicht quantifiziertes Produktivitätspotenzial. Der Stand eignet sich als standardisierte Basis für die vorgesehenen Projektfamilien, nicht als fertige Produktionsanwendung oder universelle Vorlage. Ob sich die zentrale Investition wirtschaftlich lohnt, bleibt ohne Nutzungs- und Kostendaten offen.

Kurz erklärt: Reife und Plattformabdeckung

Betriebsreife bedeutet, dass eine Anwendung unter den vorgesehenen Bedingungen zuverlässig betrieben und überwacht werden kann.

Produktionsreife geht weiter: Auch Sicherheit, Freigaben und die konkreten Anforderungen eines fertigen Produkts müssen erfüllt und nachgewiesen sein.

Plattformabdeckung beschreibt, für welche Zugangswege und Betriebssysteme eine Lösung vorgesehen und geprüft ist.

Die Fallstudie weist diese Abdeckung je nach Commit und Plattform unterschiedlich weit nach.

Das Template ist daher eine standardisierte technische Basis, aber keine fertige Produktionsanwendung.

8 Einsatzmöglichkeiten und Transfer auf Kundenprojekte

8.1 Geeignete Projekttypen

Die Eignung hängt von Zugriffskanälen, Backend und Persistenz, Deployment, nativer Integration sowie Infrastruktur-, Sicherheits- und Betriebsbedarf ab. Belegt sind generierbare Kombinationen und geprüfte Profilpfade. Die Stufen *gut geeignet*, *bedingt geeignet* und *nicht primär vorgesehen* bewerten dagegen qualitativ den Abstand zur Baseline, nicht die Erfolgswahrscheinlichkeit eines Projekts (kleiveist, 2026g, 2026j). Je mehr zentrale Anforderungen außerhalb der vorhandenen Komponenten und Nachweise liegen, desto größer sind Anpassungsbedarf und verbleibendes Risiko.

`web-only` eignet sich für browserbasierte Frontends ohne Template-Backend. `web-cloud` ergänzt zentrale API und providerneutrale Deploymentstruktur, etwa für interne Business- und Datenanwendungen. PostgreSQL bleibt optional; Authentifizierung, Berechtigungen, Fachmodelle und produktive Infrastruktur sind produktspezifisch. Damit deckt das Profil die technische Ausgangsstruktur, nicht die konkrete Geschäftsanwendung ab.

`desktop-local` passt zu lokalen Werkzeugen ohne zentrales Backend, sofern native Funktionen überschaubar ergänzt werden können; eine lokale Datenbank fehlt. `desktop-cloud` ergänzt zentrale Dienste, `full-platform` zusätzlich den Browserkanal. Der primäre Einsatzbereich sind damit Web-, Cloud- und Desktop-orientierte Business-/Full-Stack-Anwendungen. Für alle Desktopvarianten müssen weitergehende native Funktionen gezielt ergänzt und geprüft werden.

Kategorie	Projekttypen	Maßgebliches Kriterium
GUT GEEIGNET	Browser-Frontend; Web + Backend/Cloud; lokales Desktop-Werkzeug; Desktop + Cloud; Web + Desktop + Cloud	Zugriffskanäle und Laufzeitschichten entsprechen einem geprüften Profil; Fach- und Betriebsfunktionen bleiben zu ergänzen.
BEDINGT GEEIGNET	hoher nativer Anteil; stark individualisierte Infrastruktur; hohe Compliance- oder Sicherheitsauflagen	Ein Profil deckt nur einen Teil ab; erhebliche Erweiterungen oder zusätzliche Nachweise sind erforderlich.
NICHT PRIMÄR VORGESEHEN	harte Echtzeitsysteme; Game-Engine-basierte Spiele; native Mobile-Apps; Firmware- und Embedded-Kernsysteme	Die tragende Laufzeit- oder Plattformarchitektur liegt außerhalb der Vite-/FastAPI-/Tauri-Baseline.

Abbildung 17: Qualitative Eignungsübersicht nach Architekturfit und Anpassungsbedarf

Quelle: Eigene Transferbewertung auf Grundlage der Profildefinitionen und der technischen Abnahme (kleiveist, 2026g, 2026j).

Abbildung 17 und Tabelle 10 fassen die Zuordnung zusammen. *Geeignet* bedeutet nur, dass die Baseline wesentliche Plattformteile abdeckt. Fachlogik, Oberfläche, Integrationen, Produktsicherheit und Betrieb bleiben ebenso offen wie die wirtschaftliche Bewertung.

Tabelle 10: Zuordnung technischer Projekttypen zur Template-Baseline

Projekttyp	Eignung	Profil	Notwendige Ergänzungen und Grenzen
Browser-Frontend	gut	web-only	Fachlogik und UI; externe APIs möglich, aber kein FastAPI-Backend der Baseline.
Webanwendung mit API/Cloud	gut	web-cloud	Authentifizierung, Fachmodelle und produktiver Betrieb; PostgreSQL nur bei relationalem Persistenzbedarf.
lokales Desktop-Werkzeug	gut	desktop-local	Native Kommandos, Packaging und Signing produktspezifisch; keine lokale Datenbank enthalten.
Desktop mit zentralem Backend	gut	desktop-cloud	Produktive API, Security und Signing; PostgreSQL bleibt optional.
Browser + Desktop + Cloud	gut	full-platform	Kanalbezogene UX und Tests; Persistenz nur mit separater Capability.
Anwendung mit hohem nativen Anteil	bedingt	Desktopprofil oder andere Basis	Erheblicher Aufwand für Plattform-APIs, Berechtigungen, systemnahe Dienste oder native UI.
stark spezialisierte Infrastruktur	bedingt	nach Teilarchitektur	Eignung nur, wenn Client oder Einzeldienst sinnvoll abgrenzbar ist; Infrastruktur nicht vorgegeben.
Hard Real-Time, Game Engine, native Mobile- oder Embedded-Kernsysteme	nicht primär vorgesehen	kein Profil	Tragende Laufzeit- und Plattformanforderungen liegen außerhalb der Baseline.

Quelle: Eigene qualitative Zuordnung auf Grundlage von kleiveist (2026g) und kleiveist (2026j).

Kurz erklärt: Technische Basis einer Business-Anwendung

Eine Business-Anwendung unterstützt Arbeitsabläufe in einem Unternehmen, zum Beispiel die Bearbeitung und gemeinsame Nutzung von Daten.

Die Fachlogik umfasst die Regeln und Abläufe des konkreten Geschäftsbereichs.

Eine technische Ausgangsbasis stellt allgemeine Bausteine wie Oberfläche, Backend oder Deployment-Struktur bereit, auf denen ein Produkt aufgebaut wird.

Das Template liefert eine solche Basis, aber weder die Fachlogik noch die fertige Geschäftsanwendung.

8.2 Ungeeignete und eingeschränkt geeignete Projekttypen

Eingeschränkt geeignet sind Projekte mit umfangreicher Betriebssystemintegration, plattformspezifischer UI, systemnahen Diensten oder Treibern. Die Tauri-Hülle belegt dafür keine ausreichende native Basis. Auch stark verteilte Infrastruktur sowie besondere Event-Streaming- oder HPC-Anforderungen können eine andere Architektur erfordern; die Baseline bleibt höchstens für einen abgegrenzten Teil nutzbar.

Hard-Real-Time-Systeme, grafikintensive Spiele, Firmware, Embedded-Systeme und native Mobilanwendungen gehören nicht zum primären Zielbereich. Gewöhnliche Live-Aktualisierungen machen eine Business-Anwendung nicht zum Echtzeitsystem. Einzelne mobile oder hardwarenahe Funktionen sind nicht ausgeschlossen; ihre technische Basis liefert das Template jedoch nicht. Entscheidend ist daher nicht ein einzelnes Merkmal, sondern ob der technische Kern des Produkts von der Baseline getragen wird.

Hohe Sicherheits- oder Compliance-Anforderungen schließen die Baseline nicht aus, verlangen aber zusätzliche Evidenz für Schutzbedarf, Audits, Zertifizierung und Rechtskonformität. Belegt sind Compose-Validierung und Master-Images, nicht Compose-Start oder produktive Bereitstellung (GitHub, 2026a). Linux- und macOS-Pakete wurden unsigned gebaut; der Windows-Paketpfad ist fehlgeschlagen (GitHub, 2026c). Signing und Notarisierung bleiben produktspezifisch (kleiveist, 2026h). Auch die benötigten TypeScript-, Python-, Rust-, Datenbank- und Deploymentkompetenzen sind zu prüfen. Fehlen diese Nachweise oder Kompetenzen, ist das Projekt höchstens bedingt geeignet.

Kurz erklärt: Besondere Projekttypen

Native Integration bedeutet, besondere Funktionen eines Betriebssystems oder Geräts direkt zu verwenden. Bei Hard Real Time muss eine Reaktion unter allen vorgesehenen Bedingungen eine feste Zeitgrenze einhalten.

Eine Game Engine stellt spezielle Werkzeuge für grafikintensive Spiele bereit.

Ein Embedded System ist ein Rechner in einem Gerät; seine hardwarenahe Software heißt Firmware.

Eine native Mobile App wird gezielt für ein mobiles Betriebssystem entwickelt, während HPC besonders rechenintensive Aufgaben auf leistungsstarken Systemen bezeichnet.

Compliance bedeutet, verbindliche rechtliche, vertragliche oder organisatorische Vorgaben einzuhalten und dies bei Bedarf nachzuweisen.

Für Projekte mit einem solchen technischen Kern liefert das Template keine vollständige Basis; bei hohen Compliance-Anforderungen sind zusätzliche Prüfungen nötig.

8.3 Analyse von Kundenanforderungen

Die Analyse beginnt mit fachlichem Ziel, Nutzergruppen, Arbeitsabläufen und Einsatzumgebung. Erst danach werden Anforderungen technisch eingeordnet. Gemeinsame Datenbearbeitung kann etwa Backend und Persistenz erfordern, legt aber noch nicht PostgreSQL fest.

Getrennt erfasst werden Browser- und Desktopzugang, Serverfunktionen, Persistenz, Offline-Bedarf, native Funktionen, externe APIs, Betrieb, Deployment, Sicherheit und Compliance. `desktop-local` enthält trotz fehlendem Cloud-Backend weder lokales Datenmodell noch Synchronisation. Tabelle 11 bündelt die Leitfragen.

Tabelle 11: Kompakte Checkliste zur technikneutralen Anforderungsanalyse

Bereich	Leitfrage	Einfluss auf die Auswahl
Fachfunktion und Nutzung	Welche Abläufe, Nutzergruppen und Einsatzorte sind abzudecken?	Bestimmt Produktumfang; noch keine Profilfestlegung.
Zugriffskanäle	Wird das Produkt im Browser, als Desktop-Anwendung oder über beide Kanäle genutzt?	Trennt Web-, Desktop- und Full-platform-Pfade.
Zentrale Dienste	Müssen Funktionen oder gemeinsam genutzte Daten serverseitig bereitstehen?	Erfordert ein backendfähiges Cloudprofil.
Persistenz und Offline-Betrieb	Werden zentrale relationale Daten benötigt; muss der Client ohne Netz arbeitsfähig sein?	PostgreSQL nur bei Backendbedarf; lokale Speicherung und Synchronisation sind separate Ergänzungen.
Native Integration	Welche Betriebssystem-APIs, Geräte oder plattformspezifischen Oberflächen sind notwendig?	Bestimmt Tauri-Bedarf und Abstand zur nativen Baseline.
Integration und Betrieb	Welche Fremdsysteme, Zielumgebungen, Skalierungs- und Betriebsanforderungen bestehen?	Erzeugt produktspezifische Adapter und Infrastrukturentscheidungen.
Security und Compliance	Welche Schutz-, Nachweis-, Audit- oder regulatorischen Vorgaben gelten?	Erfordert zusätzliche Maßnahmen und Evidenz unabhängig vom Profil.

Quelle: Eigene Darstellung.

Das Ergebnis umfasst Plattformkomponenten und Abweichungen, etwa bei Authentifizierung, Datenmodellen, Integrationen, Observability, Backups, Infrastruktur oder Signing. Erst dann lässt sich zwischen passendem Profil, begrenzter Erweiterung und einer Lösung außerhalb der Baseline entscheiden. Die Methode folgt damit bewusst der Reihenfolge Anforderung, technische Einordnung und erst anschließend Profil- oder Technologieauswahl.

Kurz erklärt: Kundenanforderungen

Eine Kundenanforderung beschreibt, was ein Produkt leisten oder unter welchen Bedingungen es arbeiten soll.

Eine funktionale Anforderung benennt eine gewünschte Funktion; eine technische Anforderung legt beispielsweise Plattform, Sicherheit oder Betrieb fest.

Offline-Betrieb bedeutet, dass benötigte Aufgaben auch ohne Netzwerkverbindung möglich sein sollen.

Eine externe API ist eine Schnittstelle zu einem anderen System.

Monitoring und Observability helfen dabei, den Zustand und das Verhalten einer laufenden Anwendung zu beobachten und Fehler zu verstehen.

Ein Backup ist eine gesicherte Kopie, aus der Daten wiederhergestellt werden können.

Compliance-Anforderungen halten verbindliche Vorgaben fest; ein Audit ist eine nachvollziehbare Prüfung ihrer Einhaltung.

Diese Anforderungen werden vor der Profilwahl gesammelt, weil das Template nicht alle davon bereits abdeckt.

8.4 Auswahl des Projektprofils

Die Zugriffskanäle bestimmen das Profil: `web-only` steht für Browser ohne Backend, `web-cloud` für Browser mit Backend, `desktop-local` für Desktop ohne Cloud-Backend, `desktop-cloud` für Desktop mit Backend und `full-platform` für Browser und Desktop gemeinsam. Ein Profil für Backend ohne providerneutrale Deployment-Dateien existiert nicht.

Gewählt wird das kleinste passende Profil. Erst danach wird PostgreSQL als separate Achse geprüft und nur bei Bedarf zu einem Backendprofil ergänzt. Für `web-only` und `desktop-local` ist dies unzulässig (kleiveist, 2026g).

Dokumentiert wird, ob das Profil vollständig passt, benannte Erweiterungen benötigt oder zentrale Anforderungen verfehlt. Im letzten Fall sind eine andere Architektur oder klar abgegrenzte Teilnutzung zu prüfen, nicht ersatzweise das größte Profil. Teamkompetenz und wirtschaftliche Randbedingungen ergänzen diese technische Entscheidung, ändern aber nicht die Bedeutung der Profile.

Kurz erklärt: Das kleinste passende Profil

Das kleinste passende Profil enthält nur die technischen Bausteine, die ein Projekt tatsächlich benötigt. Die technische Auswahlentscheidung richtet sich zuerst danach, ob Browser, Desktop und ein Backend erforderlich sind. PostgreSQL wird anschließend getrennt geprüft und nur einem passenden Backendprofil hinzugefügt. Dieses Vorgehen vermeidet unnötige Komponenten und damit zusätzliche Komplexität. Wenn ein Profil zentrale Anforderungen nicht erfüllt, ist statt des größten Profils eine andere Architektur oder eine klar begrenzte Teilnutzung zu prüfen.

8.5 Generierung eines Kundenprojekts

`init` erzeugt aus Profil, Capability und Projektidentität ein separates Projekt. Dessen aktives Manifest heißt `project-profile.toml`. Die Generierung ist der Übergabepunkt zur Produktentwicklung, nicht die Fertigstellung des Produkts (kleiveist, 2026g, 2026l). Bei Desktopprojekten gehört eine gültige Anwendungs-ID zu diesen Eingaben.

Im Zielverzeichnis prüfen `doctor`, `install`, `run` und `test --suite all` die Baseline. Profilabhängige Tauri-, Datenbank- oder Containerprüfungen ergänzen den in Abbildung 18 gezeigten Transfer.

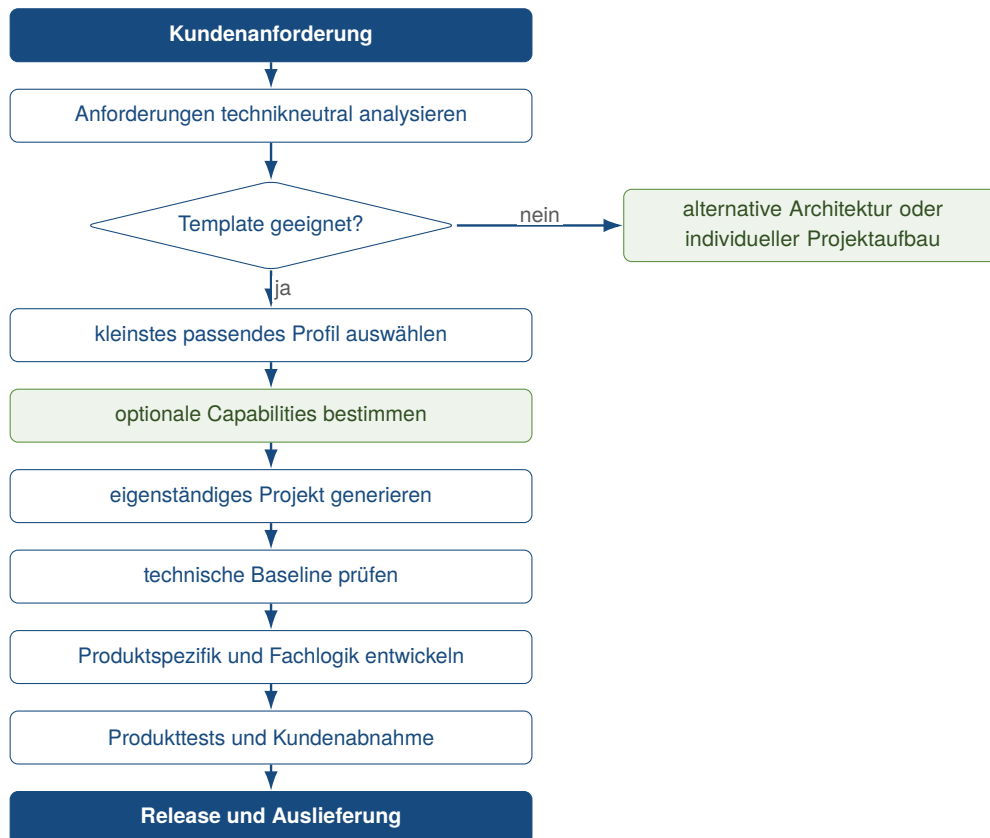


Abbildung 18: Vertikaler Transferprozess von der Kundenanforderung zum Produktprojekt

Quelle: Eigene Darstellung.

Kurz erklärt: Vom Template zum Produktprojekt

Mit der Generierung wird aus der gewählten Template-Ausstattung ein eigenständiges Projekt erstellt. Dieser Schritt ist der Übergang von der gemeinsamen technischen Vorlage zur Entwicklung des konkreten Kundenprodukts.

Das erzeugte Projekt enthält eine prüfbare Ausgangsbasis, ist aber noch kein fertiges Produkt.

Fachfunktionen, produktspezifische Anforderungen und eigene Nachweise werden erst danach im neuen Projekt ergänzt.

8.6 Überführung in die Produktentwicklung

Nach der Generierung übernimmt das eigenständige Produktprojekt Fachmodelle, Geschäftsregeln, Oberfläche, Integrationen, Daten und Betrieb. Auch Authentifizierung, Berechtigungen, Skalierung, Monitoring, Backup, Datenschutz und Signing bleiben Produktaufgaben. Die Baseline strukturiert diese Arbeit, löst sie aber nicht.

Master und Produkt besitzen getrennte Lebenszyklen. Es gibt keinen dauerhaften Update- oder Synchronisationsmechanismus. Master-Änderungen müssen gezielt in Produkte übernommen werden; Produktänderungen gehören nur nach Generalisierungs- und Verifikationsprüfung zurück in die Baseline.

Template- und Produktabnahme bleiben getrennt. Die commitgebundene Abnahme belegt Generierung, Installation, Tests und ausgewählte Builds der Baseline, nicht Deployment, Signing oder Fach- und

Sicherheitsanforderungen (kleiveist, 2026j). Das Produkt benötigt dafür eigene Tests und Nachweise. Diese müssen die konkreten Kundenanforderungen, Integrationen und Betriebsbedingungen abdecken.

Kurz erklärt: Aufgaben der Produktentwicklung

Im Produktprojekt wird die zuvor erläuterte Fachlogik für den konkreten Kunden umgesetzt.

Skalierung sorgt dafür, dass die Anwendung auch bei wachsender Nutzung genügend Leistung bereitstellen kann.

Monitoring und Backups müssen für die konkreten Betriebsbedingungen eingerichtet werden. Monitoring macht Probleme sichtbar. Backups ermöglichen die Wiederherstellung verlorener oder beschädigter Daten. Datenschutz regelt den rechtmäßigen und geschützten Umgang mit personenbezogenen Daten.

Bei der Produktabnahme wird geprüft, ob das fertige Produkt die konkreten Kundenanforderungen und Betriebsbedingungen erfüllt.

Die Abnahme der Template-Basis ersetzt diese eigene Produktprüfung nicht.

9 Schlussbetrachtung

9.1 Zusammenfassung der Ergebnisse

Das Master-Template begegnet wiederholtem Setup-Aufwand und divergierenden Projektbasen mit einem gemeinsamen Kern und deklarativer Variabilität. Es bündelt Generierung, Konfiguration, Verantwortungsgrenzen, Arbeitsabläufe und Wartung.

Fünf Profile kombinieren gemeinsames Vite-Frontend, profilabhängiges FastAPI-Backend und optionale Tauri-Schicht; PostgreSQL ist nur mit Backend wählbar. `control.py` vereinheitlicht den Entwicklungsweg. Generierte Projekte bleiben eigenständig und erhalten keine automatischen Master-Updates.

Die lokale Abnahme eines früheren Commits stützt Profil-, PostgreSQL-, Test-, Web-Build- und Linux-Paketierungspfade. Die aktuelle externe CI ist nur teilweise erfolgreich; weitere Nachweise fehlen oder sind produktspezifisch. Belegt ist damit eine kontrolliert variable Baseline, nicht Produktionsreife oder empirische Kundeneignung.

9.2 Beantwortung der Fragestellungen

Fragestellung 1: Adressierte Anforderungen. Das Template adressiert die Anforderungen durch eine generierbare Baseline, klare Komponenten, Profile, gemeinsame Workflows, zentrale Pflege und versionierte Konfiguration. Getrennte Laufzeit- und Prüfpfade stützen dies; Fachlogik, Authentifizierung, Berechtigungen und Produktivbetrieb bleiben produktspezifisch.

Fragestellung 2: Varianten für Web, Cloud und Desktop. Fünf Presets kombinieren das gemeinsame Frontend mit Backend-, Cloud- und Tauri-Bausteinen; PostgreSQL wird nur bei Backend-fähigkeit ergänzt. Generierte Projekte enthalten überwiegend benötigte Pfade. `desktop-cloud` und `full-platform` sind technisch gleich gescaffoldet und semantisch abgegrenzt.

Fragestellung 3: Wiederverwendung. Die zentrale Pflege von Implementierungen, Profilen, Generator, Konfiguration, Tests und Dokumentation fördert Wiederverwendung und einheitliche Ausgangs-

stände. Der Vorteil gilt für die Baseline und künftige Generierungen; bestehende Produktprojekte erhalten keine automatischen Updates.

Fragestellung 4: Reduktion wiederkehrender Aufwände. Automatisierte oder einheitliche Profilwahl, Generierung, Diagnose, Installation, Start, Tests und Builds schaffen plausibles Einsparpotenzial. Zeit-, Aufwands- und Vergleichsdaten fehlen jedoch; eine Produktivitätssteigerung ist unter Einbezug zentraler Pflege und mehrerer Toolchains nicht nachgewiesen.

Fragestellung 5: Projekteignung. Das Template eignet sich als Basis für verwandte Web-, Desktop- und kombinierte Business-Anwendungen mit abbildbarem Plattform-, Cloud- und Persistenzbedarf. Stark native Anwendungen, Spiele und spezialisierte Echtzeitsysteme liegen außerhalb des primären Zielbereichs. Die qualitative Einstufung ist keine universelle Eignungszusage.

Fragestellung 6: Grenzen, Wartungsaufwände und Risiken. Grenzen sind wachsende Regressionsbreite, manuelle Vertragssynchronisierung, die nicht strikt getrennte Capability-Auswahl, fehlende Produktupdates und der Pflegebedarf mehrerer Toolchains. Tooling-, Desktopprofil- und Windows-CI sind aktuell rot; Release Validation, Compose-Start und Branch Protection fehlen. Signing, Notarisierung, Updater, Produktiv-Deployment und Sicherheitsmodell bleiben Produktaufgaben.

9.3 Kritische Würdigung

Die projektnahe Fallstudie ist begrenzt generalisierbar. Dokumentation, Abnahme und lokale Prüfung stammen aus dem Projektkontext; die Entwicklerperspektive kann die Evidenzauswahl beeinflussen. Literatur und offizielle Dokumentation ersetzen keine unabhängige Replikation. Commitgebundene Befunde bleiben dadurch gültig, ihre Übertragung auf andere Stände oder Projekte aber begrenzt.

Die Evidenz ist heterogen: Einer früheren, überwiegend Linux-basierten Abnahme stehen gemischte aktuelle CI-Ergebnisse gegenüber. Ohne Coverage-Messung, Compose-Start und erfolgreiche Plattformläufe entsteht kein umfassender Qualitäts- oder Produktionsnachweis; unsignierte Pakete ersetzen keine Produktreleases.

Übertragbar sind vor allem drei Prinzipien: zentrale Baseline, deklarative Variabilität und gemeinsame Tooling-Einstiege. Produktivität und Eignung wurden nur qualitativ bewertet; Zeitmessungen, Vergleichsgruppen, Kundenfälle und Wirtschaftsdaten fehlen. Diese Lücken begrenzen insbesondere Aussagen zur tatsächlichen Wirkung im Projektalltag.

9.4 Ausblick

Kurzfristig sind die roten Tooling-, Desktopprofil- und Windows-CI-Pfade auf einem festen Commit zu korrigieren. Compose-Start mit Health und Readiness, Release Validation und Branch Protection sollen die technische Evidenz ergänzen, ohne Produktreife zu beanspruchen.

Mittelfristig benötigen konkrete Produkte Signierung, Notarisierung und einen Updater. Auch Credentials, Deployment, Authentifizierung und RBAC bleiben produktspezifisch. Für die Baseline bieten sich

drei Schritte an: gemeinsame Contracts automatisieren, Capabilities strenger wählen und generierte Projekte kontrolliert synchronisieren.

Langfristig sollte eine projektübergreifende Vergleichsstudie Onboarding, ersten Build, Setup-Fehler und Pflegeaufwand messen. Reale Kundenprojekte könnten die Profil- und Eignungskriterien prüfen.

9.5 Fazit

Das Template ist eine zentral gepflegte, profilbasierte Baseline für die definierten Web-, Cloud- und Desktopvarianten. Gemeinsamer Kern, Capabilities und Tooling vereinheitlichen wiederkehrende Projektabläufe.

Kontrollierte Variabilität schafft plausibles Produktivitätspotenzial. Seine Größe und die Kundeneignung sind ohne empirische Daten nur für den abgegrenzten Zielbereich analytisch begründbar.

Der Nachweis endet bei der untersuchten technischen Baseline. Lokale Abnahme, aktuelle CI-Fehler, offene externe Prüfungen sowie produktbezogene Release- und Sicherheitsaufgaben erlauben keine Gleichsetzung mit einem fertigen, plattformübergreifend verifizierten Produkt.

Literaturverzeichnis

- Bayer, M. (2026). *Alembic tutorial* [Alembic 1.19.1 Documentation]. Verfügbar 8. August 2026 unter <https://alembic.sqlalchemy.org/en/latest/tutorial.html>
- Clements, P. C., & Northrop, L. M. (2001). *Software product lines: Practices and patterns*. Addison-Wesley Professional. Verfügbar 8. August 2026 unter <https://www.sei.cmu.edu/library/software-product-lines-practices-and-patterns/>
- Forsgren, N., Storey, M.-A., Maddila, C., Zimmermann, T., Houck, B., & Butler, J. (2021). The space of developer productivity: There's more to it than you think. *Queue*, 19(1), 20–48. <https://doi.org/10.1145/3454122.3454124>
- GitHub. (2026a, 7. August). *Core ci, run 31168215013* [GitHub-Actions-Lauf für Commit a4487ac im Repository Template-Projekte]. Verfügbar 9. August 2026 unter <https://github.com/kleiveist/Template-Projekte/actions/runs/31168215013>
- GitHub. (2026b). *Creating a repository from a template* [GitHub Documentation]. Verfügbar 9. August 2026 unter <https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-repository-from-a-template>
- GitHub. (2026c, 7. August). *Desktop ci, run 31168214763* [GitHub-Actions-Lauf für Commit a4487ac im Repository Template-Projekte]. Verfügbar 9. August 2026 unter <https://github.com/kleiveist/Template-Projekte/actions/runs/31168214763>
- GitHub. (2026d, 7. August). *Postgresql integration, run 31168216208* [GitHub-Actions-Lauf für Commit a4487ac im Repository Template-Projekte]. Verfügbar 9. August 2026 unter <https://github.com/kleiveist/Template-Projekte/actions/runs/31168216208>
- GitHub. (2026e, 7. August). *Profile matrix, run 31168215737* [GitHub-Actions-Lauf für Commit a4487ac im Repository Template-Projekte]. Verfügbar 9. August 2026 unter <https://github.com/kleiveist/Template-Projekte/actions/runs/31168215737>
- ISO/IEC. (2023). *Systems and software engineering—systems and software quality requirements and evaluation (square)—product quality model* (International Standard Nr. ISO/IEC 25010:2023). International Organization for Standardization. Verfügbar 8. August 2026 unter <https://www.iso.org/standard/78176.html>
- Klabnik, S., Nichols, C., & Krycho, C. (2026). *The rust programming language* [Official Rust Documentation]. Verfügbar 8. August 2026 unter <https://doc.rust-lang.org/book/>
- kleiveist. (2026a, 7. August). *Central project control cli* [CLI-Implementierung im Repository Template-Projekte]. Verfügbar 9. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db896202009ff6c13567c319dfdb/tools/control.py>
- kleiveist. (2026b, 7. August). *Container builds and local production simulation* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 9. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db896202009ff6c13567c319dfdb/docs/tools/container-builds.md>
- kleiveist. (2026c, 7. August). *Continuous integration* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 9. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db896202009ff6c13567c319dfdb/docs/tools/ci.md>
- kleiveist. (2026d, 6. August). *Database feature* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfd6db896202009ff6c13567c319dfdb/docs/def/database-feature.md>

- kleiveist. (2026e, 6. August). *Deployment architecture* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/deployment-architecture.md>
- kleiveist. (2026f, 6. August). *Framework architecture* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/architecture.md>
- kleiveist. (2026g, 6. August). *Project profiles* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/project-profiles.md>
- kleiveist. (2026h, 7. August). *Release and desktop packaging model* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 9. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/tools/release-model.md>
- kleiveist. (2026i, 6. August). *Runtime configuration* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/def/configuration.md>
- kleiveist. (2026j, 7. August). *Template final acceptance* [Technisches Abnahmeprotokoll im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/dev/template-final-acceptance.md>
- kleiveist. (2026k, 7. August). *Template-projekte: Full-stack project template* [GitHub-Repository, geprüfter Commit a4487ac]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte>
- kleiveist. (2026l, 6. August). *Tooling guide* [Projektdokumentation im Repository Template-Projekte]. Verfügbar 8. August 2026 unter <https://github.com/kleiveist/Template-Projekte/blob/a4487acfcdf6db896202009ff6c13567c319dfdb/docs/tools/tooling.md>
- Krueger, C. W. (1992). Software reuse. *ACM Computing Surveys*, 24(2), 131–183. <https://doi.org/10.1145/130844.130856>
- Lamb, C., & Zacchiroli, S. (2022). Reproducible builds: Increasing the integrity of software supply chains. *IEEE Software*, 39(2), 62–70. <https://doi.org/10.1109/MS.2021.3073045>
- Microsoft. (2026, 27. Juli). *The typescript handbook* [TypeScript Documentation]. Verfügbar 8. August 2026 unter <https://www.typescriptlang.org/docs/handbook/intro.html>
- PostgreSQL Global Development Group. (2026). *Postgresql 18.4 documentation*. Verfügbar 8. August 2026 unter <https://www.postgresql.org/docs/current/index.html>
- Ramírez, S. (2026). *Fastapi features* [Official FastAPI Documentation]. Verfügbar 8. August 2026 unter <https://fastapi.tiangolo.com/features/>
- Runeson, P., & Höst, M. (2009). Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2), 131–164. <https://doi.org/10.1007/s10664-008-9102-8>
- SQLAlchemy Authors and Contributors. (2026, 15. Juni). *Engine configuration* [SQLAlchemy 2.0 Documentation, Release 2.0.51]. Verfügbar 8. August 2026 unter <https://docs.sqlalchemy.org/en/20/core/engines.html>
- Tauri Contributors. (2026a). *Capabilities* [Tauri 2 Documentation]. Verfügbar 8. August 2026 unter <https://v2.tauri.app/security/capabilities/>

Tauri Contributors. (2026b, 22. Juli). *What is tauri?* [Tauri 2 Documentation]. Verfügbar 8. August 2026 unter <https://v2.tauri.app/start/>

VoidZero Inc. and Vite Contributors. (2026). *Getting started* [Vite Documentation]. Verfügbar 8. August 2026 unter <https://vite.dev/guide/>